

CONTENIDO

PARTE UNO

MODELOS, COMPUTADORAS Y ANÁLISIS DEL ERROR 3

- PT 1.1 Motivación 3
- PT 1.2 Fundamentos matemáticos 5
- PT 1.3 Orientación 8

CAPÍTULO 1

Modelos matemáticos y solución de problemas de ingeniería 11

- 1.1 Un modelo matemático simple 11
- 1.2 Leyes de conservación e ingeniería 18
- Problemas 22

CAPÍTULO 2

Computadoras y programas 25

- 2.1 La computación y su entorno 26
- 2.2 Desarrollo de programas 27
- 2.3 Diseño de algoritmos 31
- 2.4 Composición de programas 42
- 2.5 Control de calidad 45
- 2.6 Documentación y mantenimiento 49
- 2.7 Estrategia de programación 51
- Problemas 55

CAPÍTULO 3

Aproximaciones y errores de redondeo 59

- 3.1 Cifras significativas 60
- 3.2 Exactitud y precisión 62
- 3.3 Definiciones de error 63
- 3.4 Errores de redondeo 66
- Problemas 82

CAPÍTULO 4

Serie de Taylor y errores de truncamiento 84

- 4.1 Las series de Taylor 84
- 4.2 Error de propagación 101
- 4.3 Error numérico total 106
- 4.4 Equívocos, errores de formulación e incertidumbre en los datos 108
- Problemas 109

EPÍLOGO: PARTE UNO 111

- PT 1.4 Elementos de juicio 111
- PT 1.5 Relaciones y fórmulas importantes 114
- PT 1.6 Métodos avanzados y algunas referencias adicionales 114

PARTE DOS**RAÍCES
DE ECUACIONES
119**

- PT 2.1 Motivación 119
- PT 2.2 Antecedentes matemáticos 121
- PT 2.3 Orientación 123

CAPÍTULO 5**Métodos de intervalos 127**

- 5.1 Métodos gráficos 127
- 5.2 Método de bisección 131
- 5.3 Método de la falsa posición 141
- 5.4 Búsquedas con incrementos y determinación de valores iniciales 147
- Problemas 148

CAPÍTULO 6**Métodos abiertos 150**

- 6.1 Iteración simple de punto fijo 151
- 6.2 Método de Newton-Raphson 156
- 6.3 Método de la secante 162
- 6.4 Raíces múltiples 167
- 6.5 Sistemas de ecuaciones no lineales 170
- Problemas 175

CAPÍTULO 7**Raíz de polinomios 177**

- 7.1 Polinomios en ciencia e ingeniería 177
- 7.2 Cálculo con polinomios 180
- 7.3 Métodos convencionales 184
- 7.4 Método de Müller 184
- 7.5 Método de Bairstow 188
- 7.6 Otros métodos 194
- 7.7 Localización de raíces con librerías y paquetes de cómputo 194
- Problemas 204

CAPÍTULO 8**Aplicaciones en ingeniería: raíces de ecuaciones 206**

- 8.1 Leyes de los gases ideales y no ideales (ingeniería química e ingeniería petrolera) 206
- 8.2 Flujo en un canal abierto (ingeniería civil e ingeniería ambiental) 209
- 8.3 Diseño de un circuito eléctrico (ingeniería eléctrica) 213
- 8.4 Análisis de vibraciones (ingeniería mecánica e ingeniería aeroespacial) 215
- Problemas 223

EPÍLOGO: PARTE DOS 229

- PT 2.4 Elementos de juicio 229
- PT 2.5 Relaciones importantes y fórmulas 232
- PT 2.6 Métodos avanzados y referencias adicionales 232

PARTE TRES**ECUACIONES
ALGEBRAICAS
LINEALES 235**

- PT 3.1 Motivación 235
- PT 3.2 Antecedentes matemáticos 238
- PT 3.3 Orientación 245

**CAPÍTULO 9
Eliminación de Gauss 249**

- 9.1 Resolución de pequeños conjuntos de ecuaciones 249
- 9.2 Eliminación de Gauss simple 256
- 9.3 Desventajas de los métodos de eliminación 263
- 9.4 Técnicas para mejorar las soluciones 269
- 9.5 Sistemas complejos 276
- 9.6 Sistemas de ecuaciones no lineales 277
- 9.7 Gauss-Jordan 279
- 9.8 Resumen 281
- Problemas 281

**CAPÍTULO 10
Descomposición LU e inversión de matrices 284**

- 10.1 Descomposición LU 284
- 10.2 Matriz inversa 294
- 10.3 Análisis de error y condición del sistema 298
- Problemas 306

**CAPÍTULO 11
Matrices especiales y el método de Gauss-Seidel 307**

- 11.1 Matrices especiales 307
- 11.2 Gauss-Seidel 311
- 11.3 Ecuaciones algebraicas lineales con librerías y paquetes de software 319
- Problemas 327

**CAPÍTULO 12
Aplicaciones en la ingeniería: ecuaciones algebraicas lineales 329**

- 12.1 Análisis en estado estable de un sistema de reactores (ingeniería química/petrolera) 329
- 12.2 Análisis de una estructura estáticamente determinada (ingeniería civil/ambiental) 332
- 12.3 Corrientes y voltajes en circuitos de resistores (ingeniería eléctrica) 336
- 12.4 Sistemas masa-resorte (ingeniería mecánica/aeroespacial) 338
- Problemas 341

EPÍLOGO: PARTE TRES 348

- PT 3.4 Elementos de juicio 348
- PT 3.5 Relaciones importantes y fórmulas 349
- PT 3.6 Métodos avanzados y referencias adicionales 349

PARTE CUATRO

**OPTIMIZACIÓN
353**

- PT 4.1 Motivación 353
- PT 4.2 Bases matemáticas 359
- PT 4.3 Orientación 360

CAPÍTULO 13**Optimización unidimensional no restringida 364**

- 13.1 Búsqueda de la sección dorada 365
- 13.2 Interpolación cuadrática 372
- 13.3 Método de Newton 374
- Problemas 376

CAPÍTULO 14**Optimización multidimensional sin restricciones 378**

- 14.1 Métodos directos 378
- 14.2 Métodos gradiente 383
- Problemas 397

CAPÍTULO 15**Optimización restringida 399**

- 15.1 Programación lineal 399
- 15.2 Optimización restringida no lineal 411
- 15.3 Optimización con paquetes de software 411
- Problemas 421

CAPÍTULO 16**Aplicaciones en la ingeniería: optimización 424**

- 16.1 Diseño de un tanque con el menor costo (ingeniería química/petrolera) 424
- 16.2 Mínimo costo en tratamiento de aguas de desecho (ingeniería civil/ambiental) 429
- 16.3 Máxima transferencia de potencia para un circuito (ingeniería eléctrica) 434
- 16.4 Diseño de una bicicleta de montaña (ingeniería mecánica/aeroespacial) 438
- Problemas 440

EPÍLOGO: PARTE CUATRO 446

- PT 4.4 Elementos de juicio 446
- PT 4.5 Referencias adicionales 447

PARTE CINCO

**AJUSTE DE
CURVAS 449**

- PT 5.1 Motivación 449
- PT 5.2 Antecedentes matemáticos 451
- PT 5.3 Orientación 461

CAPÍTULO 17**Regresión por mínimos cuadrados 465**

- 17.1 Regresión lineal 465
- 17.2 Regresión de polinomios 481
- 17.3 Regresión lineal múltiple 487

PART**DIFER
NUM
INTEC**

- 17.4 Forma general lineal por mínimos cuadrados 490
- 17.5 Regresión no lineal 496
- Problemas 500

CAPÍTULO 18

Interpolación 502

- 18.1 Diferencia dividida de Newton para la interpolación de polinomios 503
- 18.2 Interpolación de polinomios de Lagrange 515
- 18.3 Coeficientes de un polinomio de interpolación 519
- 18.4 Interpolación inversa 520
- 18.5 Comentarios adicionales 521
- 18.6 Interpolación segmentaria 524
- Problemas 534

CAPÍTULO 19

Aproximación de Fourier 537

- 19.1 Ajuste de curvas con funciones sinusoidales 538
- 19.2 Serie de Fourier continua 544
- 19.3 Frecuencia y dominios de tiempo 548
- 19.4 Integral y transformada de Fourier 552
- 19.5 Transformada discreta de Fourier (TDF) 554
- 19.6 Transformada rápida de Fourier 556
- 19.7 El espectro de potencia 563
- 19.8 Ajuste de curvas con librerías y paquetes 564
- Problemas 576

CAPÍTULO 20

Aplicaciones en ingeniería: ajuste de curvas 578

- 20.1 Regresión lineal y modelos de población. (ingeniería química/petrolera) 578
- 20.2 Uso de segmentarias para estimar la transferencia de calor (ingeniería civil/ambiental) 582
- 20.3 Análisis de Fourier (ingeniería eléctrica) 583
- 20.4 Análisis de datos experimentales (ingeniería mecánica/aeroespacial) 585
- Problemas 587

EPÍLOGO: PARTE CINCO 594

- PT 5.4 Elementos de juicio 594
- PT 5.5 Relaciones importantes y fórmulas 595
- PT 5.6 Métodos avanzados y referencias adicionales 597

PARTE SEIS

DIFERENCIACIÓN NUMÉRICA E INTEGRACIÓN 601

- PT 6.1 Motivación 601
- PT 6.2 Antecedentes matemáticos 611
- PT 6.3 Orientación 613

CAPÍTULO 21

Fórmulas de integración de Newton-Cotes 617

- 21.1 La regla trapezoidal 619
- 21.2 Reglas de Simpson 630

- 21.3 Integración con segmentos desiguales 640
- 21.4 Fórmulas de integración abierta 643
- Problemas 644

CAPÍTULO 22**Integración de ecuaciones 646**

- 22.1 Algoritmos de Newton-Cotes para ecuaciones 646
- 22.2 Integración de Romberg 647
- 22.3 Cuadratura de Gauss 653
- 22.4 Integrales impropias 661
- Problemas 665

CAPÍTULO 23**Diferenciación numérica 666**

- 23.1 Diferenciación de fórmulas con alta exactitud 666
- 23.2 Extrapolación de Richardson 669
- 23.3 Derivadas de datos desigualmente espaciados 671
- 23.4 Derivadas e integrales para datos con errores 672
- 23.5 Integración/diferenciación numérica con librerías y paquetes 674
- Problemas 678

CAPÍTULO 24**Aplicaciones en la ingeniería: integración numérica y diferenciación 680**

- 24.1 Integración para determinar la cantidad total de calor (ingeniería química/petrolera) 680
- 24.2 Fuerza efectiva sobre el mástil de un bote de carreras (ingeniería civil/ambiental) 682
- 24.3 Raíz media cuadrática de la corriente por integración numérica (ingeniería eléctrica) 685
- 24.4 Integración numérica para calcular el trabajo (ingeniería mecánica/aeroespacial) 687
- Problemas 691

EPÍLOGO: PARTE SEIS 698

- PT 6.4 Elementos de juicio 698
- PT 6.5 Relaciones importantes y fórmulas 699
- PT 6.6 Métodos avanzados y referencias adicionales 699

PARTE OC**ECUACION
DIFERENCI
PARCIALES**

RTE SIETE

**UACIONES
ERENCIAS
DINARIAS 703**

- PT 7.1 Motivación 703
- PT 7.2 Antecedentes matemáticos 706
- PT 7.3 Orientación 709

CAPÍTULO 25**Métodos de Runge-Kutta 714**

- 25.1 Método de Euler 715
- 25.2 Mejoras del método de Euler 728
- 25.3 Métodos de Runge-Kutta 736

25.4	Sistemas de ecuaciones	748
25.5	Métodos adaptativos de Runge-Kutta	753
	Problemas	761

CAPÍTULO 26

Métodos rígidos y de multipaso 763

26.1	Rigidez	763
26.2	Métodos multipaso	767
	Problemas	788

CAPÍTULO 27

Problemas de valores en la frontera y de valores propios 790

27.1	Métodos generales para problemas de valores en la frontera	791
27.2	Problemas de valores propios	797
27.3	EDO y valores propios con librerías y paquetes	815
	Problemas	823

CAPÍTULO 28

Aplicaciones en ingeniería: ecuaciones diferenciales ordinarias 825

28.1	Uso de EDO para analizar la respuesta transitoria de un reactor (ingeniería química/petrolera)	825
28.2	Modelos depredador-presa y caos (ingeniería civil/ambiental)	832
28.3	Simulación de corriente transitoria para un circuito eléctrico (ingeniería eléctrica)	835
28.4	El péndulo oscilante (ingeniería mecánica/aeroespacial)	841
	Problemas	845

EPÍLOGO: PARTE SIETE 849

PT 7.4	Elementos de juicio	849
PT 7.5	Relaciones y fórmulas importantes	850
PT 7.6	Métodos avanzados y referencias adicionales	850

PARTE OCHO

ECUACIONES DIFERENCIALES PARCIALES 855

PT 8.1	Motivación	855
PT 8.2	Orientación	859

CAPÍTULO 29

Diferencias finitas: ecuaciones elípticas 862

29.1	La ecuación de Laplace	862
29.2	Técnica de solución	864
29.3	Condiciones en la frontera	871
29.4	La aproximación del volumen de control	877
29.5	Software para resolver ecuaciones elípticas	880
	Problemas	881

CAPÍTULO 30

Diferencias finitas: ecuaciones parabólicas 883

30.1	Ecuación de conducción del calor	883
30.2	Métodos explícitos	884

- 30.3 Un método implícito simple 889
- 30.4 El método de Crank-Nicolson 892
- 30.5 Ecuaciones parabólicas en dos dimensiones espaciales 895
- Problemas 899

CAPÍTULO 31

Método del elemento finito 900

- 31.1 El enfoque general 901
- 31.2 Aplicación del elemento finito en una dimensión 905
- 31.3 Problemas en dos dimensiones 914
- 31.4 EDP con librerías y paquetes 918
- Problemas 927

CAPÍTULO 32

Aplicaciones en ingeniería: ecuaciones diferenciales parciales 928

- 32.1 Balance de masa en una dimensión de un reactor (ingeniería química/ petrolera) 928
- 32.2 Deflexiones de una placa (ingeniería civil/ambiental) 933
- 32.3 Problemas de campo electrostático en dos dimensiones (ingeniería eléctrica) 935
- 32.4 Solución por elemento finito a una serie de resortes (ingeniería mecánica/ aeroespacial) 938
- Problemas 942

EPÍLOGO: PARTE OCHO 944

- PT 8.3 Elementos de juicio 944
- PT 8.4 Relaciones y fórmulas importantes 945
- PT 8.5 Métodos avanzados y referencias adicionales 945

APÉNDICE A: LAS SERIES DE FOURIER 946

APÉNDICE B: EMPECEMOS CON MATHCAD 948

APÉNDICE C: EMPECEMOS CON MATLAB 958

BIBLIOGRAFÍA 967

ÍNDICE 971

MODELOS, COMPUTADORAS Y ANÁLISIS DEL ERROR

PT1.1 MOTIVACIÓN

Los métodos numéricos son técnicas mediante las cuales es posible formular problemas matemáticos de tal forma que puedan resolverse usando operaciones aritméticas. Aunque hay muchos tipos de métodos numéricos, comparten una característica común: invariablemente se debe realizar un buen número de tediosos cálculos aritméticos. No es raro que con el desarrollo de computadoras digitales eficientes y rápidas, el papel de los métodos numéricos en la solución de problemas de ingeniería haya aumentado en forma considerable en los últimos años.

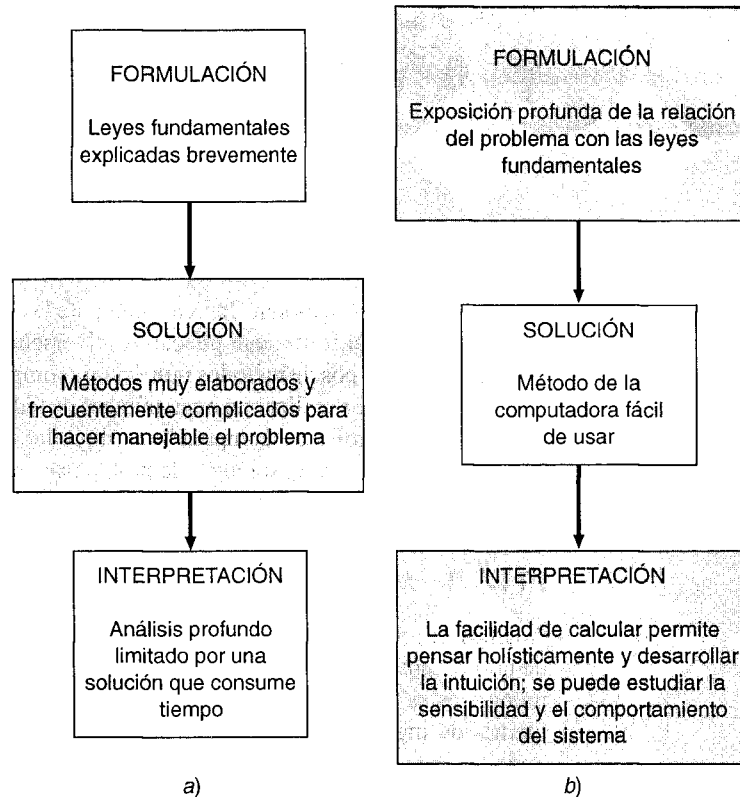
PT1.1.1 Métodos sin computadora

Además de proporcionar un aumento en la potencia de cálculo, la disponibilidad general de las computadoras (especialmente de las personales) y su asociación con los métodos numéricos ha influido de manera muy significativa en el proceso de la solución de problemas de ingeniería. Antes de la computadora los ingenieros sólo contaban con tres métodos para la solución de problemas:

1. Se encontraban las soluciones de algunos problemas usando métodos exactos o analíticos. Con frecuencia estas soluciones resultaban útiles y proporcionaban una comprensión excelente del comportamiento de algunos sistemas. Sin embargo, las soluciones analíticas sólo pueden encontrarse para una clase limitada de problemas. Éstos incluyen a los que pueden aproximarse mediante modelos lineales y también aquellos que tienen una geometría simple y pocas dimensiones. En consecuencia, las soluciones analíticas tienen valor práctico limitado porque la mayoría de los problemas reales no son de tipo lineal, e implican formas y procesos complejos.
 2. Para analizar el comportamiento de los sistemas se usaban soluciones gráficas. Estas tomaban la forma de gráficas o nomogramas; aunque las técnicas gráficas a menudo pueden emplearse para resolver problemas complejos, los resultados no son muy precisos. Además, las soluciones gráficas (sin la ayuda de una computadora) son en extremo tediosas y difíciles de implementar. Finalmente, las técnicas gráficas están limitadas a los problemas que puedan describirse usando tres dimensiones o menos.
 3. Para implementar los métodos numéricos se utilizaban calculadoras y reglas de cálculo. Aunque en teoría estas aproximaciones deberían ser perfectamente adecuadas para resolver problemas complicados, en la práctica se presentan algunas dificultades. Los cálculos manuales son lentos y tediosos. Además, los resultados no son consistentes, debido a que surgen equivocaciones cuando se efectúan las tareas de esta manera.
-

FIGURA PT1.1

Las tres fases en la solución de problemas de ingeniería en *a)* la era anterior a las computadoras y *b)* la era de las computadoras. Los tamaños de los recuadros indican el nivel de importancia que se dirige a cada fase. Las computadoras facilitan la implementación de técnicas de solución y así permiten un mayor cuidado sobre los aspectos creativos de la formulación de problemas y la interpretación de resultados.



Antes del uso de la computadora se gastaba mucha energía en la técnica misma de solución, en vez de aplicarla sobre la definición del problema y su interpretación (figura PT1.1a). Esta situación desafortunadamente se debía al tiempo y trabajo monótono que se requería para obtener resultados numéricos con técnicas que no utilizaban la computadora.

Hoy en día, las computadoras y los métodos numéricos proporcionan una alternativa para cálculos complicados. Al usar la computadora para obtener soluciones directamente, se pueden aproximar los cálculos sin tener que recurrir a suposiciones de simplificación o a técnicas lentas. Aunque las soluciones analíticas aún son muy valiosas, tanto para resolver problemas como para proporcionar una mayor comprensión, los métodos numéricos representan alternativas que aumentan en forma considerable la capacidad para confrontar y resolver los problemas; como resultado, se dispone de más tiempo para aprovechar las habilidades creativas personales. Por consiguiente, es posible dar más importancia a la formulación de un problema, a la interpretación de la solución y a su incorporación al sistema total, o conciencia “holística” (figura PT1.1b).

PT1.1.2 Los métodos numéricos y la práctica de la ingeniería

Desde finales de la década de los cuarenta, la multiplicación y la disponibilidad de las computadoras digitales han llevado a una verdadera explosión en el uso y desarrollo de

los métodos numéricos. Al principio, este crecimiento estaba limitado por el costo de las grandes computadoras (*mainframes*), por lo que muchos ingenieros seguían usando simples planteamientos analíticos en una buena parte de su trabajo. No es necesario mencionar que la reciente evolución de computadoras personales de bajo costo, ha dado un fácil acceso a mucha gente, a poderosas capacidades de cómputo. Además, existen muchas razones por las cuales se deben estudiar los métodos numéricos:

1. Los métodos numéricos son herramientas muy poderosas para la solución de problemas. Son capaces de manejar sistemas de ecuaciones grandes, no linealidades y geometrías complicadas, comunes en la práctica de la ingeniería y, a menudo, imposibles de resolver analíticamente. Por lo tanto, aumentan la habilidad de quien los estudia para resolver problemas.
2. En el transcurso de su carrera, es posible que el lector tenga la ocasión de usar *software* disponible comercialmente que contenga métodos numéricos. El uso inteligente de estos programas depende del conocimiento de la teoría básica en la que se basan estos métodos.
3. Hay muchos problemas que no pueden plantearse al emplear programas “hechos”. Si está versado en los métodos numéricos y es un adepto de la programación de computadoras, entonces tiene la capacidad de diseñar sus propios programas para resolver los problemas, sin tener que comprar un *software* costoso.
4. Los métodos numéricos son un vehículo eficiente para aprender a servirse de las computadoras. Es bien sabido que un camino efectivo para aprender programación actualmente es escribir programas de computadora. Porque la mayoría de los métodos numéricos son diseñados para implementarlos en las computadoras, por tanto, son ideales para este propósito. También hay especialistas para ilustrar el poder y las limitaciones de las computadoras. Cuando usted implemente en forma satisfactoria los métodos numéricos en computadora y aplique éstos para resolver de otra manera los problemas difíciles, usted dispondrá de una excelente demostración de cómo las computadoras pueden servir para su desarrollo profesional. Al mismo tiempo, aprenderá a reconocer y controlar los errores de aproximación que son inseparables de los cálculos numéricos a gran escala.
5. Los métodos numéricos son un medio para reforzar su comprensión de las matemáticas, ya que una de sus funciones es convertir las matemáticas superiores a operaciones aritméticas básicas, porque profundizan en los temas que de otro modo resultarían oscuros. Esta alternativa aumenta su capacidad de comprensión y entendimiento en la materia.

PT1.2 FUNDAMENTOS MATEMÁTICOS

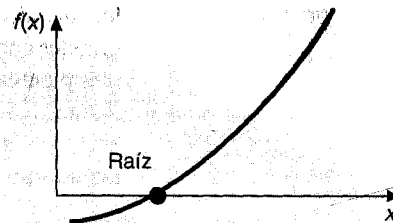
Cada parte de este libro requiere de algunos antecedentes matemáticos, por lo que el material introductorio de cada parte incluye una sección como ésta: de fundamentos matemáticos. Debido a que la parte uno, en sí, está dedicada al material básico sobre las matemáticas y la computación, en esta parte no se revisará ningún tema matemático específico. En vez de esto se presentan los temas del contenido matemático que se cubre en este libro. Estos se resumen en la figura PT1.2, y son:

FIGURA PT1.2

Resumen de los métodos que se exponen en este libro.

a) *Parte 2: Raíces de ecuaciones*

Resolver $f(x) = 0$ para x .

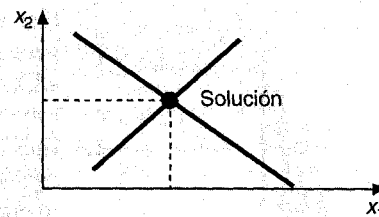
b) *Parte 3: Ecuaciones del álgebra lineal*

Dadas las a y las c resolver

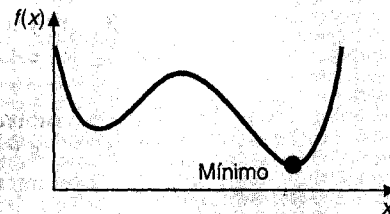
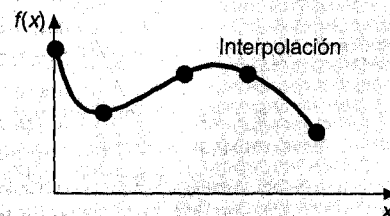
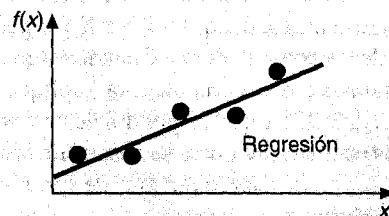
$$a_{11}x_1 + a_{12}x_2 = c_1$$

$$a_{21}x_1 + a_{22}x_2 = c_2$$

para las x .

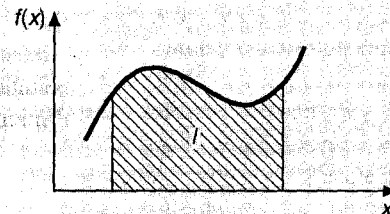
c) *Parte 4: Optimización*

Determinar la x que da el óptimo de $f(x)$

d) *Parte 5: Ajuste de curvas*e) *Parte 6: Integración*

$$I = \int_a^b f(x) dx$$

Encontrar el área bajo la curva



1. *Raíces de ecuaciones* (figura PT1.2a). Estos problemas están relacionados con el valor de una variable o de un parámetro que satisface una ecuación. Son especialmente valiosos en proyectos de ingeniería, donde con frecuencia resulta imposible despejar de manera analítica los parámetros de ecuaciones de diseño.
2. *Sistemas de ecuaciones algebraicas lineales* (figura PT1.2b). En esencia, estos problemas son similares a los de raíces de ecuaciones en el sentido de que están relacio-

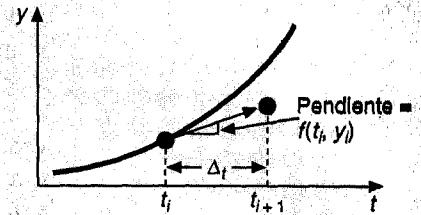
FIGURA PT1.2
(Conclusión)**f) Parte 7: Ecuación diferencial ordinaria**

Dada

$$\frac{dy}{dt} \approx \frac{\Delta y}{\Delta t} = f(t, y)$$

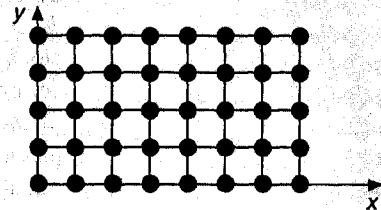
resolver para y con una función de t .

$$y_{i+1} = y_i + f(t_i, y_i) \Delta t$$

**g) Parte 8: Ecuación diferencial parcial**

Dada

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = f(x, y)$$

resolver para u como una función de x y y 

nados con valores que satisfacen ecuaciones. Sin embargo, a diferencia de satisfacer una sola ecuación, se busca un conjunto de valores que satisfaga simultáneamente un conjunto de ecuaciones algebraicas. Las ecuaciones lineales simultáneas surgen en el contexto de una variedad de problemas y en todas las disciplinas de la ingeniería. En particular, se originan a partir de modelos matemáticos de sistemas grandes de elementos interrelacionados, como: estructuras, circuitos eléctricos y redes de flujo; aunque también pueden encontrarse en otras áreas de los métodos numéricos como el ajuste de curvas y ecuaciones diferenciales.

3. **Optimización** (figura PT1.2c). Esos problemas involucran la determinación de un valor o valores de una variable independiente que correspondan al “mejor” o al óptimo valor de una función. Así, como en la figura PT1.2c, concierne a la identificación máxima y mínima. Tales problemas ocurren generalmente en el contexto del diseño de ingeniería. Esto también surge en otros métodos numéricos. Nosotros nos dirigiremos a ambos, o sea, a la optimización con restricción de una o varias variables, que también describe la optimización restringida con énfasis particular en la programación lineal.
4. **Ajuste de curvas** (figura PT1.2d). A menudo se presentará la oportunidad de ajustar curvas a un conjunto de datos representados por puntos. Las técnicas que se han desarrollado para este fin pueden dividirse en dos categorías generales: regresión e interpolación. La primera se emplea cuando hay un grado significativo de error asociado a los datos; con frecuencia los datos experimentales son de esta clase. Para estas situaciones, la estrategia es encontrar una curva que represente la tendencia general de los datos sin necesidad de tocar los puntos individuales. En contraste, la interpolación se maneja cuando el objetivo es determinar valores intermedios entre datos que estén, relativamente libres de error. Tal es el caso de la información tabulada. Para estas situaciones, la estrategia es ajustar una curva directamente mediante los puntos y usar esta curva para predecir valores intermedios.
5. **Integración** (figura PT1.2e). Tal como se representa, una interpretación física de la integración numérica es la determinación del área bajo la curva. La integración tie-

ne muchas aplicaciones en la práctica de la ingeniería, empezando por la determinación de los centroides de objetos con formas extravagantes, hasta el cálculo de cantidades totales basadas en conjuntos de medidas discretas. Adicionalmente, las fórmulas de integración numérica juegan un papel importante en la solución de las ecuaciones diferenciales.

6. *Ecuaciones diferenciales ordinarias* (figura PT1.2f). Éstas tienen un enorme significado en la práctica de la ingeniería. Esto se debe a que muchas leyes físicas están expresadas en términos de la razón de cambio de una cantidad más que en términos de su magnitud. Entre los ejemplos se observan desde los modelos de predicción demográfico (razón de cambio de la población) hasta la aceleración de un cuerpo en descenso (razón de cambio de la velocidad). Dos tipos de problemas son direccionados: problemas con valor inicial y valores en la frontera. Además de cubrir la computación de valores propios.
7. *Ecuaciones diferenciales parciales* (figura PT1.2g). Las ecuaciones diferenciales parciales son usadas para caracterizar sistemas de ingeniería, donde el comportamiento de la cantidad física es expresada en términos de rapidez de cambio con respecto a dos o más variables independientes. Se incluyen ejemplos de estado establecido de distribución de temperaturas sobre una placa caliente (dos dimensiones espaciales) o de la variable tiempo de la temperatura de una barra caliente (tiempo y una dimensión espacial). Dos diferencias fundamentales de aproximación son empleadas para resolver numéricamente ecuaciones diferenciales parciales. En el presente texto haremos énfasis en el método de las diferencias finitas con aproximación en una forma de puntos discretos (figura PT1.2g). No obstante, también presentaremos una introducción al método del elemento finito, el cual usa una aproximación de piezas discretas.

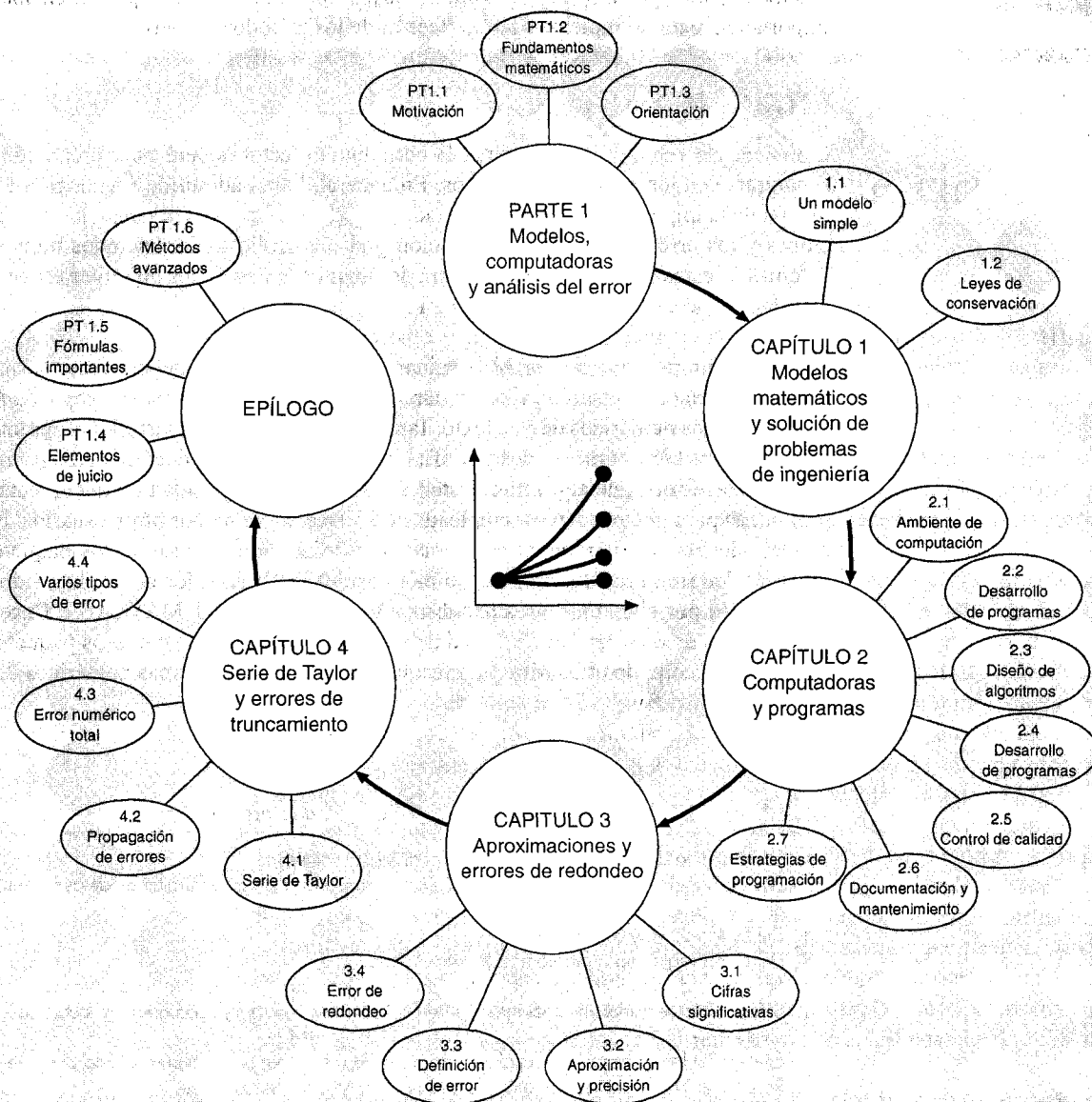
PT1.3 ORIENTACIÓN

Resulta útil esta orientación antes de proceder a la introducción de los métodos numéricos. Lo que sigue está pensado como una vista panorámica del material contenido en la parte uno. Se incluyen, además, algunos objetivos como ayuda para concentrar el esfuerzo del lector en el estudio del material.

PT1.3.1 Alcance y contenido

La figura PT1.3 es una representación esquemática del material contenido en la parte uno. Se ha elaborado este diagrama para dar un panorama global de esta parte del libro. Se considera que un sentido de “imagen global” resulta importante para desarrollar una verdadera comprensión de los métodos numéricos. Al leer un texto es posible que se pierda uno en los detalles técnicos. Siempre que el lector perciba que está perdiendo la “imagen global” vuelva a la figura PT1.3 para orientarse nuevamente. Cada parte de este libro incluye una figura similar.

La figura PT1.3 también sirve como una breve revisión previa del material que se cubre en la parte uno. El capítulo 1 está diseñado para orientarle en los métodos numéricos y para motivarlo mostrándole cómo pueden usarse estas técnicas en el proceso de elaborar modelos matemáticos aplicados a la ingeniería. El capítulo 2 es una introducción y

**FIGURA PT1.3**

Esquema de la organización del material para la parte uno: Modelos, computadoras y análisis del error.

un repaso de los aspectos de computación que están relacionados con los métodos numéricos y presenta las habilidades de programación que se deben adquirir para explotar de manera eficiente la computadora. Los *capítulos 3 y 4* se ocupan del importante tema del análisis de error, que debe entenderse bien para el uso efectivo de los métodos numéricos.

Además, se incluye un epílogo que introduce los elementos de juicio que tienen una gran importancia para la implementación efectiva de los métodos numéricos.

PT1.3.2 Metas y objetivos

Objetivos de estudio. Al terminar la parte uno el lector deberá estar preparado para aventurarse en los métodos numéricos. En general, habrá adquirido una noción fundamental de la importancia de las computadoras y el papel que desempeñan las aproximaciones y los errores en la implementación y el desarrollo de los métodos numéricos. Además de estas metas generales, deberá dominar cada uno de los objetivos específicos de estudio que se enuncian en la tabla PT1.1.

Objetivos en computación. Al terminar la parte uno, usted deberá tener suficiente experiencia en computación y desarrollar sus habilidades para hacer su propio *software* de los métodos numéricos de este texto. También será capaz de desarrollar programas de computadoras bien estructurados y confiables con base en el pseudocódigo, diagramas de flujo u otras formas de algoritmo. Usted deberá tener la capacidad de documentar sus programas para que puedan ser empleados en forma eficiente por otros usuarios. Finalmente, además de sus propios programas, usted deberá estar usando los paquetes de *software* de este libro. Los métodos numéricos del TOOLKIT, los cuales han sido proporcionados por el texto y otros paquetes conocidos, Mathcad, MATLAB o Excel son los ejemplos de dicho *software*. Usted deberá estar familiarizado con esos paquetes, ya que será más cómodo utilizarlos para resolver después los problemas numéricos de este texto.

TABLA PT1.1 Objetivos de estudio específicos para la parte uno.

1. Reconocer la diferencia entre soluciones analíticas y numéricas.
2. Entender cómo las leyes de la conservación son empleadas para desarrollar modelos matemáticos en sistemas físicos.
3. Definir top-down y diseño modular.
4. Definir las reglas para programación estructurada.
5. Ser capaz de construir programas estructurados y modulares en un lenguaje de alto nivel.
6. Saber cómo se traducen los diagramas de flujo estructurado y pseudocódigo en lenguaje de alto nivel.
7. Empezar a familiarizarse con cualquier paquete de *software* que usará en conjunto con este texto.
8. Reconocer la diferencia entre error de truncamiento y error de redondeo.
9. Entender los conceptos de cifras significativas, exactitud y precisión.
10. Reconocer la diferencia entre error relativo ϵ_r , error relativo aproximado ϵ_a y error aceptable ϵ_s y entender cómo ϵ_a y ϵ_s son usadas para terminar un cálculo iterativo.
11. Saber cuántos números son representados en computadora digital y cómo esa representación induce error de redondeo. En particular, conocer la diferencia entre precisión simple y extendida.
12. Reconocer cómo el cálculo aritmético puede introducir y amplificar el error de redondeo en calculadoras. En particular, apreciar el problema de cancelación por sustracción.
13. Saber cómo la Serie de Taylor y su residuo se emplea para representar funciones continuas.
14. Conocer la relación entre diferencias finitas divididas y derivadas.
15. Ser capaz de analizar cómo los errores se propagan en las relaciones entre funciones.
16. Estar familiarizado con los conceptos de estabilidad y condicionamiento.
17. Familiarizarse con los elementos de juicio que se describen en el epílogo de la parte uno.

CAPÍTULO 1

Modelos matemáticos y solución de problemas de ingeniería

El conocimiento y entendimiento son prerequisites para la aplicación eficaz de cualquier herramienta. Si no sabemos cómo funcionan las herramientas, tendremos serios problemas para reparar un carro, aunque la caja de herramientas sea de lo más completa.

Ésta es una realidad, particularmente cuando se usan computadoras para resolver problemas de ingeniería. Aunque tienen una gran utilidad potencial, las computadoras son prácticamente ineficientes si no se comprende el funcionamiento de los sistemas de ingeniería.

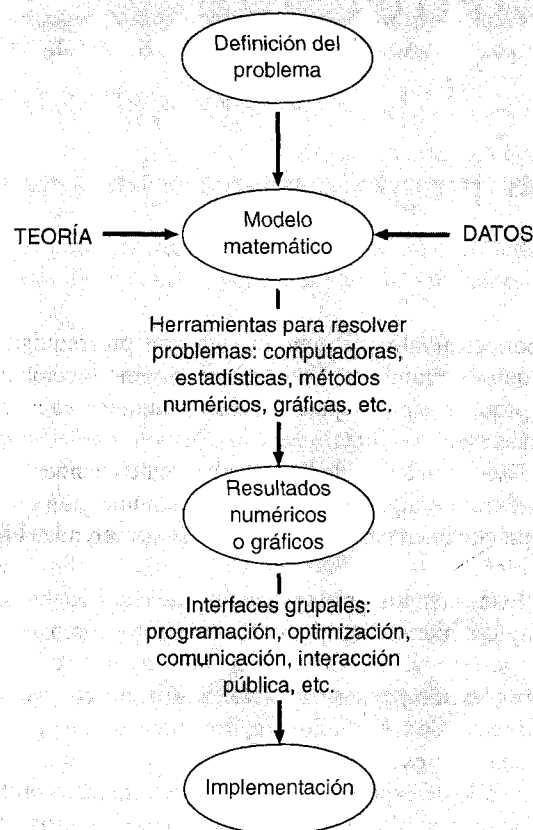
Este entendimiento es inicialmente empírico —es decir, se adquiere por la observación y la experimentación—. Sin embargo, como esta información obtenida de manera empírica es apenas esencial, sólo estamos a la mitad del camino. A través de muchos años de observación y experimentación, los ingenieros y los científicos han advertido que ciertos aspectos de sus estudios empíricos ocurren una y otra vez. Este comportamiento general puede expresarse como una ley fundamental que engloba, en esencia, el conocimiento acumulado de experiencias pasadas. Así, muchos problemas de ingeniería se resuelven mediante el empleo de un doble enfoque: el empirismo y el análisis teórico (figura 1.1).

Hay que hacer un esfuerzo para que estos dos elementos del conocimiento lleguen a fundirse en uno solo. Mientras se toman nuevas medidas, se pueden modificar las generalizaciones o descubrirse otras nuevas. De igual manera, las generalizaciones pueden tener una gran influencia en la experimentación y la observación. En lo particular, las generalizaciones pueden servir como principios de organización que pueden utilizarse para sintetizar los resultados de observaciones y experimentos en un sistema consistente y comprensible, con conclusiones que pueden ser graficadas. Desde la perspectiva de un problema de ingeniería ya resuelto, este sistema es aún más útil cuando el problema se expresa en forma similar a un modelo matemático.

El primer objetivo de este capítulo es introducir al lector en los modelos matemáticos y el papel de éstos en la solución de problemas de ingeniería. Se mostrará también la forma en que los métodos numéricos figuran en el proceso.

1.1 UN MODELO MATEMÁTICO SIMPLE

Un *modelo matemático* puede ser definido, con amplitud, como una formulación o una ecuación que expresa las características esenciales de un sistema físico o proceso en términos matemáticos. En términos generales, el modelo puede ser representado mediante una relación funcional de la forma:

**FIGURA 1.1**

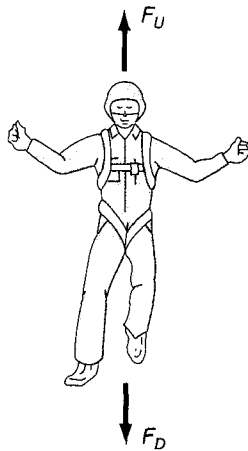
Proceso de solución de problemas de ingeniería.

$$\text{Variable dependiente} = f \left(\begin{array}{l} \text{variables} \\ \text{independientes} \end{array}, \text{parámetros}, \begin{array}{l} \text{funciones} \\ \text{de fuerza} \end{array} \right) \quad (1.1)$$

donde la *variable dependiente* es una característica que generalmente refleja el comportamiento o estado de un sistema; las *variables independientes* son, generalmente, dimensiones tales como tiempo y espacio, a través de las cuales el comportamiento del sistema será determinado; los *parámetros* son reflejo de las propiedades o la composición del sistema; y las *funciones de fuerza* las cuales son influencias externas que actúan sobre él.

La expresión matemática de la ecuación (1.1) va desde una simple relación algebraica hasta un enorme y complicado grupo de ecuaciones diferenciales. Por ejemplo, en la base de sus observaciones, Newton formuló su segunda ley del movimiento, la cual establece que la razón de cambio del *momentum* con respecto al tiempo de un cuerpo, es igual a la fuerza resultante que actúa sobre él. La expresión matemática o el modelo de la segunda ley es la ya conocida ecuación

$$F = ma \quad (1.2)$$

**FIGURA 1.2**

Representación de las fuerzas que actúan sobre un paracaidista en descenso. F_D es la fuerza hacia abajo debida a la atracción de la gravedad. F_U es la fuerza debida a la resistencia del aire.

donde F es la fuerza neta que actúa sobre el cuerpo (N, o kg m/s^2), m es la masa del objeto (kg) y a es su aceleración (m/s^2).

La segunda ley puede ser reconstruida según el formato de la ecuación (1.1), dividiendo ambos lados entre m para obtener

$$a = \frac{F}{m} \quad (1.3)$$

donde a es la variable dependiente que refleja el comportamiento del sistema, F es la función de fuerza y m es un parámetro que representa una propiedad del sistema. Note que en este caso sencillo no existe variable independiente porque aún no se predice cómo varía la aceleración con respecto al tiempo o al espacio.

La ecuación (1.3) tiene varias características típicas de los modelos matemáticos del mundo físico:

1. Describe un sistema o proceso natural en términos matemáticos.
2. Representa una idealización y una simplificación de la realidad. Es decir, ignora los detalles insignificantes del proceso natural y se concentra en sus manifestaciones elementales. Por eso, la segunda ley de Newton no incluye los efectos de la relatividad, que tienen una importancia mínima cuando se aplican a objetos y fuerzas que interactúan sobre o alrededor de la Tierra a velocidades en escalas visibles a los seres humanos.
3. Finalmente, conduce a resultados predecibles y, en consecuencia, puede emplearse con propósitos de predicción. Por ejemplo, dada la fuerza aplicada sobre un objeto y su masa, puede usarse la ecuación (1.3) para calcular la aceleración.

Debido a que se tiene una forma algebraica sencilla, la solución de la ecuación (1.2) puede despejarse directamente. Sin embargo, otros modelos matemáticos de fenómenos físicos pueden ser mucho más complejos, de modo que, o no se pueden resolver con exactitud, o requieren para su solución mayor complejidad de las técnicas matemáticas que una simple álgebra. Para ilustrar un modelo más complicado de esta clase, puede usarse la segunda ley de Newton si se quiere determinar la velocidad final de la caída libre de un cuerpo cerca de la superficie de la Tierra. Nuestro cuerpo de caída será el de un paracaidista (figura 1.2). Un modelo para este caso puede ser desarrollado usando la expresión de aceleración como la razón de cambio de la velocidad con respecto al tiempo (dv/dt), y sustituyendo esto en la ecuación (1.3) se tiene

$$\frac{dv}{dt} = \frac{F}{m} \quad (1.4)$$

donde v es la velocidad (m/s). Así, la masa multiplicada por la razón de cambio de la velocidad con respecto al tiempo es igual a la fuerza neta que actúa sobre el objeto. Si la fuerza neta es positiva, el objeto acelerará. Si es negativa, el objeto desacelerará. Si la fuerza neta es igual a cero, la velocidad del objeto se mantendrá constante.

Ahora expresemos a la fuerza neta en términos de variables y parámetros mensurables. Para un cuerpo que cae a distancias cercanas a la Tierra (figura 1.2), la fuerza total está compuesta por dos fuerzas contrarias: la atracción hacia abajo debida a la gravedad F_D y la fuerza hacia arriba debida a la resistencia del aire F_U .

$$F = F_D + F_U \quad (1.5)$$

Si a la fuerza hacia abajo se le asigna un signo positivo, se puede usar la segunda ley de Newton para formular la fuerza debida a la gravedad como

$$F_D = mg \quad (1.6)$$

donde g es la constante de gravedad, o la aceleración debida a la gravedad, que es aproximadamente igual a 9.8 m/s^2 .

La resistencia del aire puede formularse de varias maneras. Una aproximación sencilla es suponer que es linealmente proporcional a la velocidad,¹ como en

$$F_U = -cv \quad (1.7)$$

donde c es una constante de proporcionalidad llamada *coeficiente de resistencia* o *arrastre* (kg/s). Así, cuanto mayor es la velocidad de caída, mayor es la fuerza hacia arriba debida a la resistencia del aire. El parámetro c toma en cuenta las propiedades del objeto descendente, como la forma o la aspereza de su superficie, que afectan la resistencia del aire. En este caso, c podría ser una función del tipo de traje o la orientación usada por el paracaidista durante la caída libre.

La fuerza total es la diferencia entre las fuerzas hacia abajo y las fuerzas hacia arriba. Por tanto, las ecuaciones (1.4) a (1.7) pueden combinarse para dar

$$\frac{dv}{dt} = \frac{mg - cv}{m} \quad (1.8)$$

o simplificando el lado derecho de la igualdad.

$$\frac{dv}{dt} = g - \frac{c}{m}v \quad (1.9)$$

La ecuación (1.9) es un modelo que relaciona la aceleración de un cuerpo que cae con las fuerzas que actúan sobre él. Se trata de una *ecuación diferencial* porque está escrita en términos de la razón de cambio diferencial (dv/dt) de la variable que nos interesa predecir, razón por la cual a veces se denomina ecuación en diferencias. Sin embargo, en contraste con la solución dada por la segunda ley de Newton en la ecuación (1.3), la solución exacta de la ecuación (1.9) para la velocidad del paracaidista que cae no puede obtenerse mediante simples manipulaciones algebraicas u operaciones aritméticas. Más bien, es necesario usar técnicas más avanzadas, tal como el cálculo, para obtener una solución exacta o analítica. Por ejemplo, si el paracaidista está inicialmente en reposo ($v = 0$ en $t = 0$), se puede usar el cálculo para resolver la ecuación (1.9), así

$$v(t) = \frac{gm}{c} (1 - e^{-(c/m)t}) \quad (1.10)$$

Note que la ecuación (1.10) es una versión de la forma general de la ecuación (1.1), donde $v(t)$ es la variable dependiente, t es la variable independiente, c y m son parámetros y g es la función de fuerza.

¹ En efecto, la relación es actualmente no lineal y podría ser representada por una relación con potencias como $F_U = -cv^2$. Al final de este capítulo, investigaremos, en un ejercicio, de qué manera estas no linealidades influyen en el modelo.

EJEMPLO 1.1 Solución analítica al problema del paracaidista que cae

Enunciado del problema. Un paracaidista con una masa de 68.1 kg salta de un globo aerostático fijo. Aplíquese la ecuación (1.10) para calcular la velocidad antes de abrir el paracaídas. El coeficiente de resistencia es de aproximadamente 12.5 kg/s.

Solución: Al sustituir los valores de los parámetros en la ecuación (1.10) se obtiene

$$v(t) = \frac{9.8(68.1)}{12.5} (1 - e^{-(12.5/68.1)t}) = 53.39(1 - e^{-0.18355t})$$

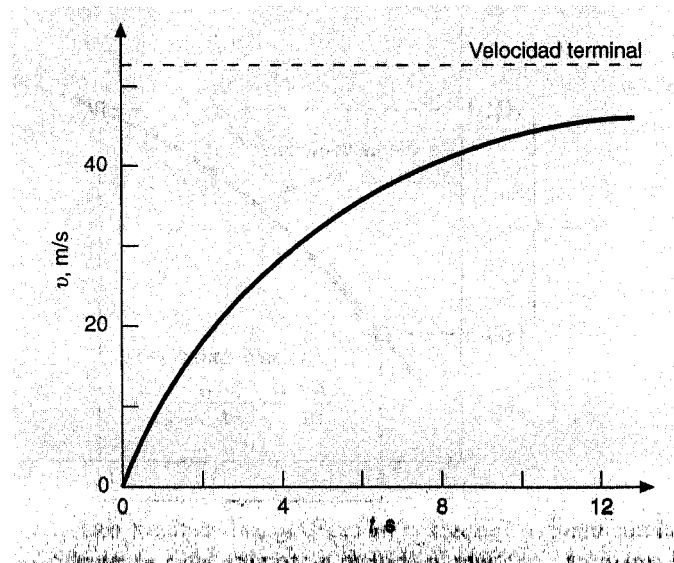
que puede ser usada para calcular

t, s	$v, m/s$
0	0.00
2	16.40
4	27.77
6	35.64
8	41.10
10	44.87
12	47.49
∞	53.39

De acuerdo con el modelo, el paracaidista acelera rápidamente (figura 1.3). Se llega a una velocidad de 44.87 m/s (100.4 mi/h) después de 10 s. Nótese también que después de un tiempo suficientemente grande alcanza una velocidad constante (o *velocidad terminal*) de 53.39 m/s (119.4 mi/h). Esta velocidad es constante porque después de un tiempo la fuerza de gravedad estará en equilibrio con la resistencia del aire. Por lo tanto, la fuerza total es cero y cesa la aceleración.

FIGURA 1.3

Solución analítica al problema del paracaidista que cae según se calcula en el ejemplo 1.1. La velocidad aumenta con el tiempo y se aproxima asintóticamente a una velocidad terminal.



A la ecuación (1.10) se le llama *solución analítica* o *exacta* porque satisface con exactitud la ecuación diferencial original. Desafortunadamente, hay muchos modelos matemáticos que no pueden resolverse con exactitud. En muchos casos, la alternativa consiste en desarrollar una solución numérica que se aproxime a la solución exacta.

Como ya se mencionó, los *métodos numéricos* son aquellos en los que se formula el problema matemático para que se pueda resolver mediante operaciones aritméticas. Esto puede ilustrarse para la segunda ley de Newton, al aproximar a la razón de cambio de la velocidad con respecto al tiempo (figura 1.4):

$$\frac{dv}{dt} \cong \frac{\Delta v}{\Delta t} = \frac{v(t_{i+1}) - v(t_i)}{t_{i+1} - t_i} \quad (1.11)$$

donde Δv y Δt son diferencias en la velocidad y el tiempo calculadas sobre intervalos finitos, $v(t_i)$ es la velocidad en el tiempo inicial t_i , y $v(t_{i+1})$ es la velocidad algún tiempo más tarde t_{i+1} . Observe que $dv/dt \cong \Delta v/\Delta t$ es aproximado porque Δt es finito. Recordando de cálculo que

$$\frac{dv}{dt} = \lim_{\Delta t \rightarrow 0} \frac{\Delta v}{\Delta t}$$

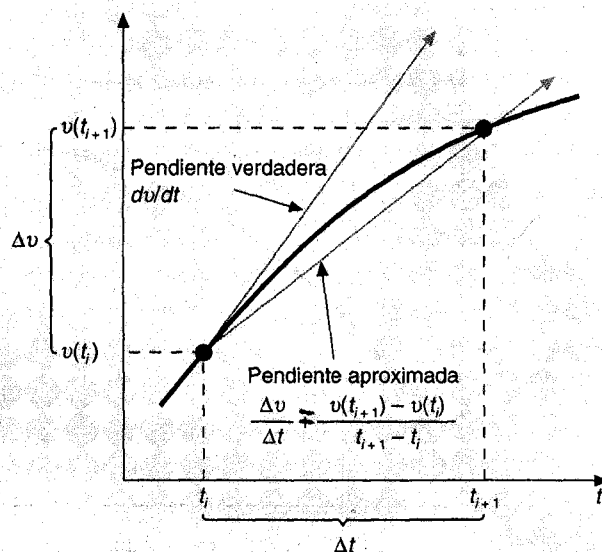
La ecuación (1.11) representa el proceso inverso.

La ecuación (1.11) se denomina una *diferencia finita dividida* y es una aproximación a la derivada en el tiempo t_i . Sustituyendo en la ecuación (1.9) da

$$\frac{v(t_{i+1}) - v(t_i)}{t_{i+1} - t_i} = g - \frac{c}{m} v(t_i)$$

FIGURA 1.4

Uso de diferencias finitas para aproximar la primera derivada de v con respecto a t .



Esta ecuación se puede reordenar para dar

$$v(t_{i+1}) = v(t_i) + \left[g - \frac{c}{m} v(t_i) \right] (t_{i+1} - t_i) \quad (1.12)$$

Nótese que el término en corchetes que está del lado derecho es la propia ecuación diferencial [ecuación (1.9)]. Esto es, da un significado al cálculo de la velocidad de cambio o la pendiente de v . Así, la ecuación diferencial ha sido transformada en una ecuación que puede ser usada para determinar algebraicamente la velocidad en t_{i+1} , usando la pendiente y los valores anteriores de v y t . Si se da un valor inicial para la velocidad en algún tiempo t_i , se puede calcular fácilmente la velocidad en un tiempo posterior t_{i+1} . Este nuevo valor de la velocidad en t_{i+1} puede utilizarse para calcular la velocidad en t_{i+2} y así sucesivamente. Es decir, a cualquier tiempo.

Nuevo valor = viejo valor + pendiente $\times$ tamaño del paso

EJEMPLO 1.2 Solución numérica al problema de la caída de un paracaidista

Enunciado del problema. Efectuar el mismo cálculo que en el ejemplo 1.1 pero usando la ecuación (1.12) para calcular la velocidad. Emplee un tamaño de paso de 2 s.

Solución. Al principio de los cálculos ($t_i = 0$), la velocidad del paracaidista es igual a cero. Con esta información y los valores de los parámetros del ejemplo 1.1, se puede usar la ecuación (1.12) para calcular la velocidad en $t_{i+1} = 2$ s.

$$v = 0 + \left[9.8 - \frac{12.5}{68.1} (0) \right] 2 = 19.60 \text{ m/s}$$

Para el siguiente intervalo (de $t = 2$ a 4 s), se repite el cálculo con el resultado

$$v = 19.60 + \left[9.8 - \frac{12.5}{68.1} (19.60) \right] 2 = 32.00 \text{ m/s}$$

Se continúa con los cálculos de manera similar para obtener valores adicionales, como en la tabla siguiente:

t, s	$v, \text{m/s}$
0	0.00
2	19.60
4	32.00
6	39.85
8	44.82
10	47.97
12	49.96
∞	53.39

Los resultados se grafican en la figura 1.5, junto con la solución exacta. Como es evidente, la solución por un método numérico se aproxima bastante bien a la solución

exacta. Sin embargo, debido a que se emplean segmentos de rectas para aproximar una función que es continuamente curva, hay algunas diferencias entre los dos resultados. Una forma de reducir estas diferencias consiste en usar un menor intervalo de cálculo. Por ejemplo, aplicar la ecuación (1.12) en intervalos de 1-s redundaría en un error menor, ya que los segmentos de recta estarían un poco más cerca de la solución verdadera. De hacer cálculos manuales, el esfuerzo asociado al usar incrementos cada vez más pequeños haría poco prácticas tales soluciones numéricas. Sin embargo, con la ayuda de una computadora personal se puede efectuar fácilmente un gran número de cálculos; por tanto, se puede modelar con exactitud la velocidad del paracaidista que cae, sin tener que resolver exactamente la ecuación diferencial.

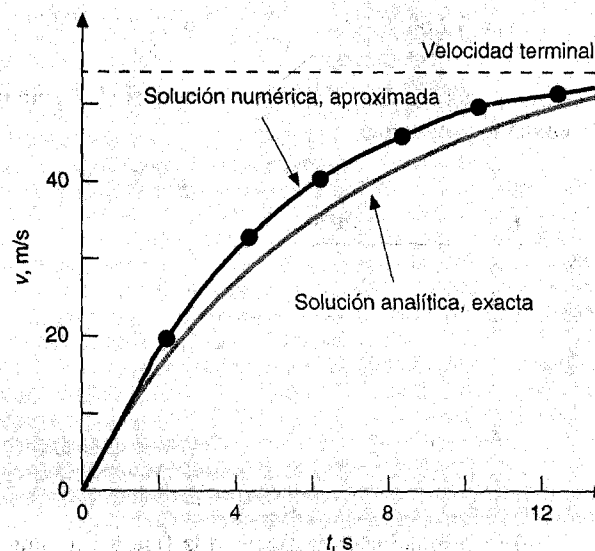
Como se vio en el ejemplo anterior, se paga un costo computacional para obtener un resultado numérico más preciso. Cada partición a la mitad del tamaño de paso para lograr más precisión nos lleva a duplicar el número de cálculos. Por consiguiente, sabemos que existe un trueque entre la exactitud y la cantidad de operaciones y el tiempo de procesamiento. Este tipo de criterios son de gran importancia en los métodos numéricos y constituyen un tema importante en este libro. En consecuencia, se ha dedicado el epílogo de la Parte Uno para hacer una introducción a más elementos de juicio de este tipo.

1.2 LEYES DE CONSERVACIÓN E INGENIERÍA

Aparte de la segunda ley de Newton, existe una mayor organización de principios en ingeniería. Entre los más importantes están las leyes de conservación. Aunque éstas son

FIGURA 1.5

Comparación de las soluciones numéricas y analíticas para el problema del paracaidista que cae.



fundamentales en una variedad de complicados y eficaces modelos matemáticos, las grandes leyes de la conservación en la ciencia y la ingeniería son conceptualmente fáciles de entender. Éstos se pueden reducir a

$$\text{Cambio} = \text{incremento} - \text{decremento} \quad (1.13)$$

Esta es precisamente la forma que nosotros empleamos cuando usamos la segunda ley de Newton para desarrollar una fuerza de equilibrio en la caída del paracaidista [ecuación (1.8)].

Pese a su sencillez, la ecuación (1.13) comprende uno de los principios fundamentales en que las leyes de conservación son usadas en ingeniería—esto es, predecir cambios con respecto al tiempo—. Nosotros le daremos a la ecuación (1.13) el nombre especial cálculo de *variable-tiempo* (o *transitivo*).

Aparte de la predicción de cambios, las leyes de la conservación se aplican en casos en que el cambio no existe. Si el cambio es cero, la ecuación (1.3) será

$$\text{Cambio} = 0 = \text{incremento} - \text{decremento}$$

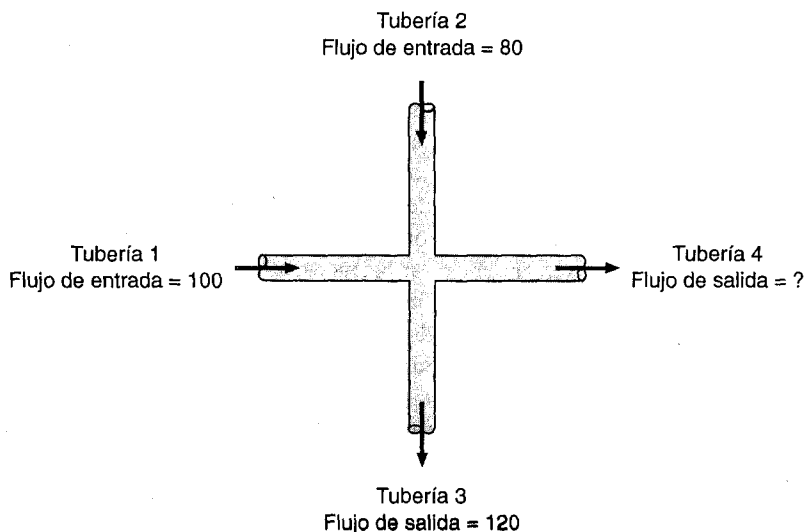
o bien,

$$\text{Incremento} = \text{decremento} \quad (1.14)$$

Así, de no ocurrir el cambio, el incremento y el decremento deberán estar en equilibrio. Este caso, que tiene una denominación especial—cálculo de *estado estacionario*—tiene muchas aplicaciones en ingeniería. Por ejemplo, para el estado continuo de flujo de

FIGURA 1.6

Balance de flujo para un fluido incompresible en estado estacionario en una unión de tuberías.



fluido incompresible en tuberías, el flujo que entra en conjunto debe estar en balance con el flujo de salida, esto es

$$\text{Flujo de entrada} = \text{flujo de salida}$$

Para la unión de tuberías de la figura 1.6, esta ecuación de balance se puede emplear para calcular el flujo de salida de la cuarta tubería que debe ser de 60.

Para la caída del paracaidista, las condiciones del estado estacionario deberían corresponder al caso en que la fuerza neta fue igual a cero o [ecuación (1.8) con $dv/dt = 0$]

$$mg = cv \quad (1.15)$$

Así, en el estado estacionario, las fuerzas hacia abajo y hacia arriba están balanceadas y la ecuación (1.15) puede ser resuelta para la velocidad terminal.

$$v = \frac{mg}{c}$$

Aunque las ecuaciones (1.13) y (1.14) pueden parecer triviales, comprenden los dos principios fundamentales en que las leyes de la conservación se emplean en ingeniería. Como tales, en capítulos subsiguientes serán parte importante de nuestros esfuerzos por mostrar la relación entre los métodos numéricos y la ingeniería. Nuestro primer medio para hacer esta relación son las aplicaciones a la ingeniería que aparecen al final de cada parte del libro.

En la tabla 1.1 se resumen algunos de los modelos sencillos en ingeniería y su relación con las leyes de conservación que serán la forma básica de muchas de esas aplicaciones. La mayoría de aplicaciones de ingeniería química harán énfasis en el equilibrio de masa para reactores. El equilibrio de la masa se deriva de la conservación de la masa. Específicamente, el cambio de masa de un compuesto químico en un reactor depende de la cantidad de masa que fluye menos el cambio de masa de salida.

Las aplicaciones en ingeniería civil y mecánica se enfocarán sobre modelos desarrollados para la conservación del *momentum*. Con respecto a la ingeniería civil, fuerzas en equilibrio se utilizan en el análisis de estructuras como las armaduras sencillas de la tabla 1.1. El mismo principio se puede aplicar en ingeniería mecánica, con el fin de analizar el movimiento transitorio superior e inferior, o las vibraciones de un automóvil.

Finalmente, las aplicaciones en ingeniería eléctrica emplean balance de corriente y energía para el modelo de circuitos eléctricos. El balance de corriente, el cual resulta de la conservación de carga, es similar en espíritu al equilibrio del flujo representado en la figura 1.6. Así como el flujo debe equilibrarse en las uniones de tuberías, la corriente eléctrica debe estar balanceada o en equilibrio en las uniones de alambres eléctricos. La conservación de la energía especifica que la suma algebraica de los cambios de voltaje alrededor de cualquier malla de un circuito deben ser igual a cero. Las aplicaciones en ingeniería pretenden ilustrar cómo se emplean actualmente los métodos numéricos para solucionar problemas de ingeniería. Como tales, dichos métodos nos permitirán examinar los métodos prácticos (tabla 1.2) que surgen en aplicaciones del mundo real. El establecer la relación entre técnicas matemáticas como los métodos numéricos y la práctica de la ingeniería es un paso decisivo para mostrar su verdadero potencial. Examinar cuidadosamente las aplicaciones a la ingeniería nos ayudará a establecer esta relación.

TABLA 1.1 Dispositivos y tipos de balance que se usan comúnmente en las cuatro grandes áreas de la ingeniería. En cada caso se especifica la ley de conservación en que se basa el balance.

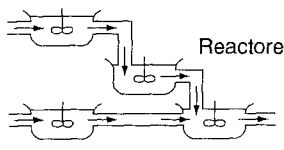
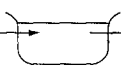
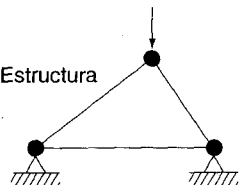
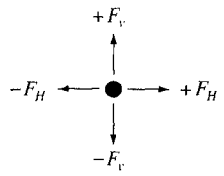
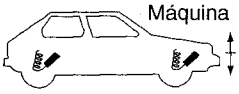
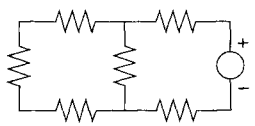
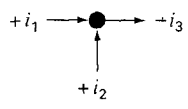
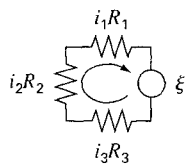
Campo	Dispositivo	Principio aplicado	Expresión matemática
Ingeniería química	 Reactores	Conservación de la masa	Balance de la masa: Entrada  Salida En un periodo $\Delta \text{masa} = \text{salidas} - \text{entradas}$
Ingeniería civil	 Estructura	Conservación del momentum	Balance de fuerzas  Para cada nodo $\sum \text{fuerzas horizontales } (F_H) = 0$ $\sum \text{fuerzas verticales } (F_v) = 0$
Ingeniería mecánica	 Máquina	Conservación del momentum	Balance de fuerzas: $\uparrow \text{ Fuerza hacia arriba}$ $\downarrow \text{ Fuerza hacia abajo}$ $x = 0$ $m \frac{d^2 x}{dt^2} = \text{Fuerza hacia abajo} - \text{fuerza hacia arriba}$
Ingeniería eléctrica	 Circuito	Conservación de carga	Balance de corriente: Para cada nodo $\sum \text{corriente } (i) = 0$ 
		Conservación de la energía	Balance del potencial  Alrededor de cada ciclo $\sum \text{fems} - \sum \text{caída de potencial en los resistores} = 0$ $\sum \xi - \sum iR = 0$

TABLA 1.2 Algunos aspectos prácticos que se investigarán en las aplicaciones a la ingeniería al final de cada parte del libro.

1. *No lineal contra lineal.* Mucho de la ingeniería clásica depende de la linealización que permite soluciones analíticas. Aunque esto es apropiado, puede ganar cierto reconocimiento cuando se revisan los problemas no lineales.
2. *Grandes sistemas contra pequeños.* Sin una computadora, no siempre se pueden examinar sistemas con más de tres componentes interactivos. Con las computadoras y métodos numéricos, se pueden examinar en forma más realista los sistemas multicomponentes.
3. *No ideal contra ideal.* En ingeniería abundan las leyes idealizadas. A menudo, hay alternativas no idealizadas que son más realistas pero que demandan muchos cálculos. La aproximación numérica puede acercarse fácilmente para aplicaciones de esas relaciones no ideales.
4. *Sensibilidad de análisis.* Debido a que están involucrados, muchos cálculos manuales requieren una gran cantidad de tiempo y esfuerzo para su implementación exitosa. Algunas veces desalienta el análisis para implementación de múltiples cálculos que son necesarios al examinar cómo un sistema responde en diferentes condiciones. Tal análisis de sensibilidad se facilita cuando los métodos numéricos permiten que la computadora asuma la carga de cómputo.
5. *Diseño.* Es a menudo una proposición sencilla para la determinación del comportamiento de un sistema como una función de parámetros. Usualmente es más difícil resolver el problema a la inversa, es decir, determinar los parámetros cuando se especifica el comportamiento requerido. A menudo, los métodos numéricos y las computadoras permiten realizar esta tarea de manera eficiente.

PROBLEMAS

1.1 Conteste falso o verdadero

- a) El valor de una variable que satisface una simple ecuación se llama raíz de una ecuación.
- b) Las diferencias finitas divididas son usadas para representar derivadas en términos aproximados.
- c) En la era anterior a las computadoras, los métodos numéricos se empleaban constantemente porque requerían un pequeño esfuerzo de cálculo.
- d) La interpolación se emplea para problemas de ajuste de curvas cuando hay un error significativo en los datos.
- e) Los modelos matemáticos no se pueden usar con propósitos de predicción.
- f) Los grandes sistemas de ecuaciones, las no linealidades y las geometrías complicadas son comunes en la práctica de la ingeniería y son fáciles de resolver analíticamente.
- g) La segunda ley de Newton es un buen ejemplo de que muchas leyes físicas se basan en la razón de cambio de las cantidades y no en sus magnitudes.
- h) Una interpretación física de la integración es el área bajo la curva.
- i) Los métodos numéricos son aquellos en los que se reformula un problema matemático para que pueda resolverse mediante operaciones aritméticas.
- j) Hoy en día se puede prestar mayor atención a la formulación e interpretación de los problemas porque las compu-

tadoras y los métodos numéricos facilitan la solución de problemas de ingeniería.

1.2 Lea las siguientes descripciones de problemas y señale qué área de los métodos numéricos —según lo señalado en la figura PT1.2— se relaciona con su solución.

- a) Supongamos que usted tiene la responsabilidad de determinar los flujos en una gran red de tuberías interconectadas entre sí para distribuir gas natural a una serie de comunidades diseminadas en un área de 51.77 m^2 (20 mi^2).
- b) Usted está haciendo experimentos para determinar la caída de voltaje a través de una resistencia como una función de la corriente. Hace las mediciones de la caída de voltaje para diferentes valores de la corriente. Aunque hay algún error asociado con sus datos, al graficar los puntos, éstos le sugieren una relación ligeramente curvilínea. Usted debe deducir una ecuación que caracterice esta relación.
- c) Usted debe desarrollar un sistema de amortiguamiento para un auto de carreras. Puede usar la segunda ley de Newton para tener una ecuación que ayude a predecir la razón de cambio en la posición de la rueda delantera en respuesta a fuerzas externas. Debe calcular el movimiento de la rueda, como una función del tiempo después de golpear contra un tope de 15.24 cm (6 pulg) a 67.041 m/s (150 mi/h).
- d) Usted tiene que calcular cuál es el ingreso anual que se va requerir en 20 años, para la construcción de un centro de entretenimiento solicitado por un cliente. El préstamo pue-

de hacerse con una tasa de interés de 10.37%. Aunque, para hacer este cálculo, la información se encuentra en tablas de economía, sólo aparecen listados los valores para tasas de interés de 10 y 11%.

- e) Usted debe determinar la distribución de temperatura en dos dimensiones de una superficie con empaque como una función de temperatura en los extremos.
- f) Para el problema del paracaidista que cae, se debe decidir el valor del coeficiente de arrastre para que un paracaidista de 889.6 N (200 lb) de peso no exceda los 160.9 km/h (100 mi/h) después de 10 s de haber saltado. Esta evaluación se hará con base en la solución analítica [ecuación (1.10)]. La información se empleará en el diseño de ropa especial para paracaidistas.
- g) Usted es experto en el trazo de caminos y debe determinar el área de un campo que está sobre dos carreteras, y medir el flujo.

1.3 Proporcione el ejemplo de un problema de ingeniería donde pueda utilizar cada uno de los siguientes tipos de métodos numéricos. De ser posible, remítase a sus propias experiencias en cursos, conferencias u otras prácticas profesionales que haya acumulado hasta la fecha.

- a) Raíces de ecuaciones
- b) Ecuaciones lineales algebraicas
- c) Ajuste de curvas: regresión e interpolación
- d) Optimización
- e) Integración
- f) Ecuaciones diferenciales ordinarias
- g) Ecuaciones diferenciales parciales

1.4 ¿Qué es la aproximación de dos puntas para resolver problemas en ingeniería? Dentro de esta categoría, ¿debería haber un lugar para las leyes de conservación?

1.5 ¿Cuál es la forma de la ley conservativa transitiva? ¿Es estado estacionario?

1.6 La siguiente información de una cuenta de banco está disponible:

Fecha	Depósitos	Retiros	Balance
5/1			512.33
	220.13	327.26	
6/1			
	216.80	378.61	
7/1			
	350.25	106.80	
8/1			
	127.31	450.61	
9/1			

Use la conservación del efectivo para calcular el balance de 6/1, 7/1, 8/1 y 9/1. Muestre cada paso en el cálculo. Este cálculo ¿es de estado estacionario o transitivo?

1.7 Dé ejemplos de leyes de conservación en ingeniería y en la vida diaria.

1.8 Vea sus libros de texto de ingeniería y busque cuatro casos en que los modelos matemáticos se usen para describir el comportamiento de los sistemas físicos. Tome nota de las variables dependientes e independientes, así como de los parámetros y las funciones de fuerza.

1.9 Compruebe que la ecuación (1.10) es la solución de la ecuación (1.9).

1.10 Repita el ejemplo 1.2. Calcule la velocidad para $t = 12$ s, con un tamaño de paso de a) 1 y b) 0.5 s. ¿Puede usted hacer cualquier juicio de apreciación del error de cálculo basado en los resultados?

1.11 Más que la relación lineal de la ecuación (1.7), la fuerza superior en la caída del paracaidista es en realidad no lineal y podría representarse mejor por una relación de potencias como

$$F_U = -c'v^2$$

donde $c' = a$ un coeficiente de arrastre de segundo orden (kg/m). Usando la relación, repita el cálculo del ejemplo 1.2 con los mismos valores de las condiciones iniciales y los parámetros. Use un valor de 0.23 kg/m para c' .

1.12 La cantidad de contaminante radiactivo uniformemente distribuido en un reactor cerrado está medida por concentraciones c (becquerel/litro, o Bq/L). El contaminante decrece a una velocidad de caída proporcional a esa concentración; esto es

$$\text{Velocidad de caída} = -kc$$

donde $k = a$ una constante por unidad de día⁻¹. Por lo tanto, de acuerdo con la ecuación (1.13), el balance de la masa para el reactor puede representarse como

$$\frac{dc}{dt} = -kc$$

$$\left(\begin{array}{c} \text{cambio} \\ \text{de masa} \end{array} \right) = \left(\begin{array}{c} \text{decremento} \\ \text{por decaimiento} \end{array} \right)$$

Use los métodos numéricos para resolver la ecuación de $t = 0$ a 1 d, con $k = 0.1$ d⁻¹. Y emplee un tamaño de paso de $\Delta t = 0.1$. La concentración de $t = 0$ es 10 Bq/L.

1.13 El tanque de almacenamiento contiene un líquido de altura y , donde $y = 0$ cuando el tanque está medio lleno. El líquido es aislado en un flujo constante con una demanda Q . El contenido es reabastecido con un cambio sinusoidal de $3Q \sin^2 t$ (véase la figura P1.13).

Para este sistema, la ecuación (1.13) puede ser representada como

$$\frac{d(Ay)}{dt} = 3Q \sin^2 t - Q$$

$$\left(\begin{array}{c} \text{cambio} \\ \text{de volumen} \end{array} \right) = (\text{flujo de entrada}) - (\text{flujo de salida})$$

o dado que la superficie del área A es constante

$$\frac{dy}{dt} = 3 \frac{Q}{A} \sin^2 t - \frac{Q}{A}$$

Use uno de los métodos numéricos para resolver para la altura y de $t = 0$ a 5 d con un tamaño de paso de 0.5 d. Los valores de los parámetros son $A = 1\,200 \text{ m}^2$ y $Q = 400 \text{ m}^3/\text{d}$.

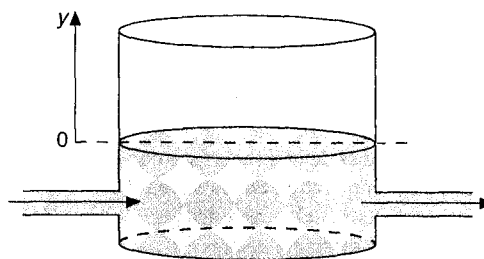


FIGURA P1.13

CAPÍTULO 2

Computadoras y programas

Los métodos numéricos combinan dos de las más importantes herramientas en el repertorio de la ingeniería: las matemáticas y las computadoras, de ahí que puedan ser definidos en general como las matemáticas por computadora. Una buena habilidad para programar computadoras podría aumentarse con el estudio de los métodos numéricos. En particular, la capacidad y limitaciones de las técnicas numéricas se aprecian mejor cuando se usan estos métodos, uno tras otro, con la computadora, para resolver problemas de ingeniería.

Usted podrá usar este libro para preparar su propio *software*. Al incrementar sus habilidades en las computadoras personales y en los dispositivos de memoria magnética, este *software* puede ser fácilmente utilizado y mejor aplicado durante su carrera. Por lo tanto, el primer objetivo de este texto es que usted llegue a tener un *software* útil y de alta calidad.

Además, los ingenieros usan cada vez más los paquetes de *software* para aplicar los métodos numéricos. Por ello, otro de los objetivos de este libro será familiarizarlo con las capacidades, operaciones y limitaciones de algunos paquetes. Además, las aplicaciones avanzadas con dichos paquetes implican a menudo alguna programación en forma de *macros* o *argumentos*. Así, aunque en apariencia pudiera parecer que el uso del *software* es lo mismo que “coser y cantar”, los ingenieros más refinados deben saber también cómo llevar a cabo programas cortos y bien estructurados para explotar con provecho esas herramientas.

Este libro incluye algunas características para que usted pueda desarrollar al máximo sus posibilidades. Primero, las técnicas numéricas se acompañan por material relacionado con su implementación efectiva en computadoras. Se proporcionan los *algoritmos* de muchos métodos. Segundo, se describen algunos de los más populares paquetes de *software* y de librerías, y se usan para resolver problemas relacionados con la ingeniería. Finalmente, está disponible el *software* suplementario para su aplicación o ilustración en varios de los métodos más elementales considerados en este libro. Compatible con varias computadoras personales, este *software* puede servir como punto de arranque para formar su propia colección de programas.

Si el lector pretende utilizar este texto como base para desarrollar sus propios programas, este capítulo proporciona información de apoyo que puede ser de utilidad, pues está escrito bajo la premisa de que se tiene alguna idea sobre programación. En vista de que este libro no pretende servir de apoyo en un curso de programación, se estudian únicamente los aspectos relacionados con la creación de *software* de alta calidad para métodos numéricos. Asimismo, se pretende dar un criterio adecuado para evaluar la calidad de sus esfuerzos.

2.1 LA COMPUTACIÓN Y SU ENTORNO

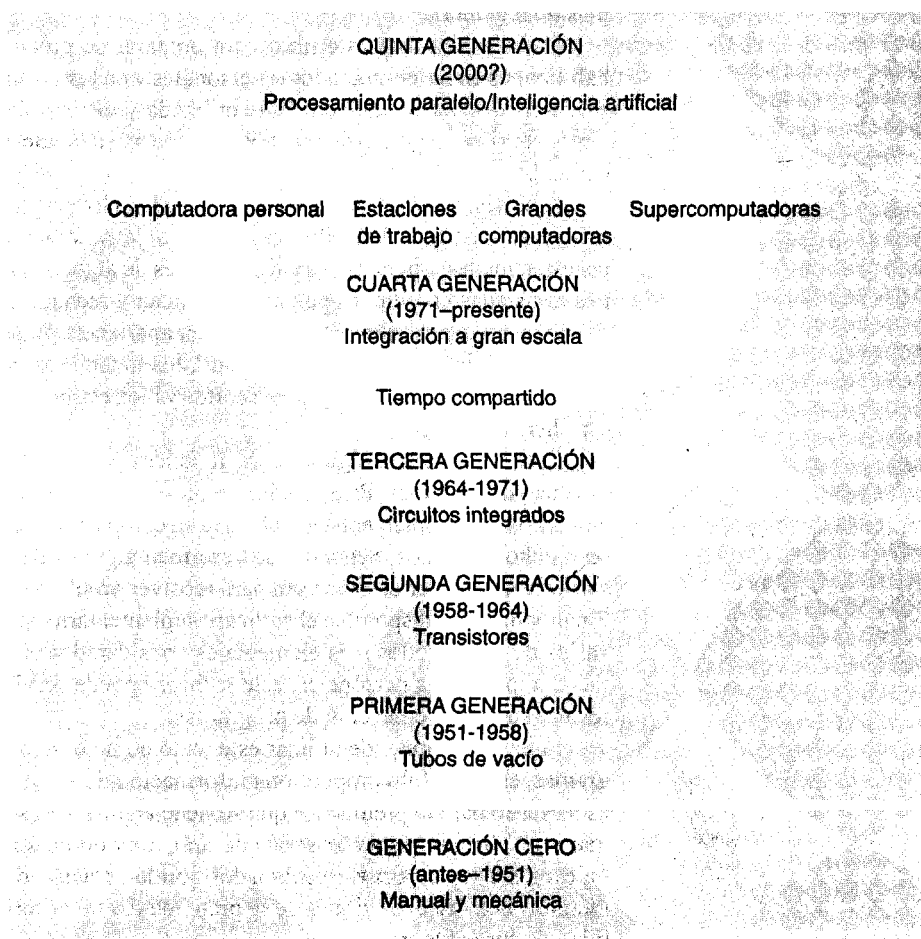
En nuestra opinión, la perspectiva histórica es muy útil para apreciar los recientes avances en el diseño de programas. Como se muestra en la figura 2.1, la edad de la moderna computadora puede ser conceptualizada como una progresión de generaciones. Antes de los años setenta, las computadoras eran monolíticas en el sentido de que eran sistemas de máquinas grandes pero no había otra opción. Como resultaba muy costoso adquirirlas, operarlas y darles mantenimiento, aquellas computadoras sólo podían ser adquiridas por grandes organismos como empresas, universidades, agencias de gobierno y el ejército. Como era de esperarse, la centralización en que estaba configurada la computación tenía un profundo impacto en la forma con que los ingenieros y los científicos interactuaban con las máquinas. En particular, había una separación clara entre el usuario y la computadora.

Al final de la década de los sesenta, la introducción de circuitos integrados dio origen a máquinas significativamente más rápidas, económicas y pequeñas. Esta ruptura tecnológica en turno generó un gran número de adelantos divergentes de la evolución anterior en la industria de la computación.

FIGURA 2.1

En función de los avances tecnológicos, la evolución de la computadora moderna puede considerarse como una sucesión de generaciones. La tercera generación diverge con el avance rápido de máquinas diseñadas para necesidades individuales. En la actualidad, está disponible un amplio rango de computadoras.

Fuente: Utilizado con el permiso de Steven Chapra y Raymond Canale, *Introducción a la computación para ingenieros*, McGraw-Hill, Nueva York, 1994.



Tomando en consideración que las computadoras estaban claramente al alcance de grandes instituciones y organizaciones, la cuarta generación dio por resultado máquinas diseñadas convenientemente para las necesidades individuales. En consecuencia, el ingeniero de hoy tiene una amplia configuración de herramientas que puede utilizar en la computación. En términos de tamaño, costo y número de usuarios, estas herramientas pueden ser clasificadas en cuatro grandes grupos: computadoras personales, estaciones de trabajo, computadoras grandes y supercomputadoras.

Como su nombre lo dice, la *computadora personal* (o PC) es una pequeña máquina cuyo principal propósito es tener un solo usuario. Las estaciones de trabajo son generalmente más poderosas y, aunque a menudo están dedicadas a un solo usuario, forman parte de una red de computadoras. Las grandes computadoras son máquinas mucho más grandes y costosas que usualmente pertenecen a una institución que da servicio a varios usuarios. Finalmente, la *supercomputadora* es grande y muy costosa, es una máquina poderosa que se usa, sobre todo, para aplicaciones muy complejas por un selecto grupo de usuarios.

Hay que hacer notar que esta clasificación no es estricta. Por ejemplo, muchas computadoras personales se han empleado como terminales, la frontera entre las PC de hoy y las estaciones de trabajo ya no es muy clara.

Cualquiera que sea la definición que se use, lo bueno de la situación actual es que se tiene acceso potencial a todo tipo de computadoras. Por ejemplo, el trabajo preliminar en una computadora de gran escala puede ser realizado por una computadora personal. Así, el enlace con las telecomunicaciones puede usarse para cargar un programa a una gran computadora, a fin de realizar una más rápida ejecución. Por este camino, se aprovecha la fuerza de cada tipo de computadora. Los ingenieros que tengan acceso a una variedad de máquinas tendrán una enorme capacidad para utilizar sus construcciones.

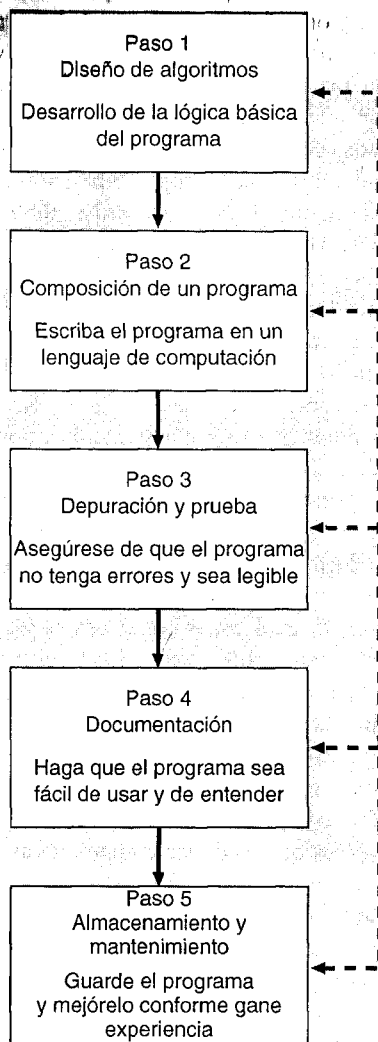
Por esta razón, y porque nos adelantamos al uso de diferentes tipos de máquinas para aprender los métodos numéricos, hemos procurado que este libro sea compatible con todos estos sistemas. Por lo tanto, mucho de este material puede ser llevado a la práctica con una variedad de sistemas que va de las computadoras personales a las grandes computadoras.

Al mismo tiempo, hemos tratado en lo posible de reconocer e investigar las interesantes nuevas aplicaciones que están directamente relacionadas con las computadoras personales. En particular, hemos consagrado una parte significativa de esta edición a los paquetes de software orientados a la PC. Creemos que el aspecto "personal" de la revolución de las microcomputadoras es el gran principio de la conmoción actual, sobre computación, entre los estudiantes de la especialidad y los profesionistas afines.

2.2 DESARROLLO DE PROGRAMAS

No importa cuál sea su tipo, una computadora es de utilidad si se le proporcionan las instrucciones con todo cuidado. Dichas instrucciones son el *software*. En la siguiente sección encontrará una gran cantidad de procesos con los cuales realizar su propio software de alta calidad para poner en práctica los métodos numéricos.

En la figura 2.2 se describen los cinco pasos que constituyen el proceso. Cada uno de ellos será tratado en las secciones subsecuentes. No obstante, antes de entrar en materia, debemos mencionar algunos cambios cualitativos que han ocurrido en el desarrollo de *software* y su gran relevancia en la aplicación de los métodos numéricos en la computadora.

**FIGURA 2.2**

Los cinco pasos necesarios para producir programas de alto nivel. Las flechas de retroalimentación indican que los primeros cuatro pasos se pueden mejorar conforme se gane experiencia.

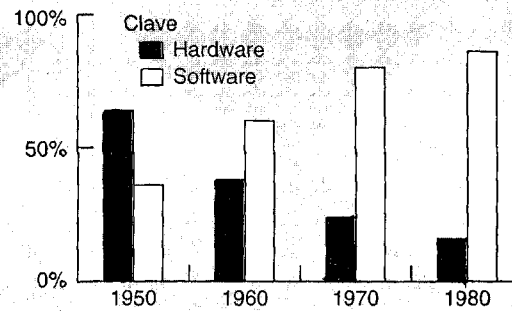
2.2.1 Estilos de programación

Así como el *hardware* ha tenido una asombrosa evolución, el proceso de desarrollo de *software* también ha pasado por una transformación radical en años recientes. En los primeros años de la computación, la programación fue más un arte que una ciencia. Aunque todos los lenguajes tenían una sintaxis y vocablos muy precisos, no había definiciones modelo de un buen estilo de programación. En consecuencia, la gente creaba sus propios criterios para constituir lo que sería una excelente pieza de *software*.

En principio, estos criterios fueron fuertemente influenciados por la disponibilidad de *hardware*. Las antiguas computadoras eran muy costosas y lentas, y tenían poca me-

FIGURA 2.3

La tendencia de costos en *hardware* y *software* en la industria de la computación. Fuente: L. M. Branscomb, "Electronics and Computers: An Overview", *Science*, 1982, 215:755.



moria. En consecuencia, uno podía hacer un buen programa si éste utilizaba tan poca memoria como fuera posible. Otra posibilidad era la ejecución rápida. Otra, más, era qué tan rápido podía escribirse por primera vez.

En consecuencia los programas eran muy diferentes. En particular, debido a que había una fuerte influencia por las limitaciones del *hardware*, los programadores tenían una pequeña recompensa en la facilidad con que se usaban los programas y en lo sencillo que era darles mantenimiento. Así, si usted necesita utilizar efectivamente y dar mantenimiento a grandes programas durante largos periodos, tiene que conservar por lo general al programador original. En otras palabras, el programa y el programador conforman un paquete de gran valor.

Hoy, muchas de las limitaciones del *hardware* ya no existen o rápidamente desaparecen. Además, desde que el costo del *software* constituye una gran parte proporcional del presupuesto total de la computadora (figura 2.3), la creación de *software* fácil de usar y modificar es un privilegio. En particular, se insiste mucho en la claridad y la amenidad, antes que en un código breve y poco claro. Este énfasis tiene un gran significado en ingeniería, donde a menudo el trabajo es realizado en equipo y los programas deben ser compartidos y modificados por varias personas.

Sistemáticamente, las ciencias de la computación han estudiado los factores y procedimientos necesarios para crear *software* de alta calidad de esta clase. Aunque los métodos resultantes están un tanto orientados hacia la programación de productos en gran escala, muchas de las técnicas generales son también útiles para el tipo de programas que los ingenieros desarrollan por rutina en su trabajo. En términos generales, a dichas técnicas les llamaremos de *programación y diseño estructurado*. En esta sección trataremos tres de esas propuestas: diseño modular, diseño top-down, programación estructurada.

2.2.2 Diseño modular

Imagine usted qué difícil sería estudiar un libro de texto que no tuviera capítulos, ni secciones ni párrafos. Podríamos dividir las tareas o los temas en partes más pequeñas, de manera que fuesen más fáciles de manejar; con esa misma tendencia, podríamos dividir los programas de computadora en pequeños subprogramas o *módulos* que puedan ser preparados y puestos a prueba por separado. A esta forma de trabajar se le llama *diseño modular*.

La característica más importante de cada uno de los módulos es que éstos sean tan independientes entre sí como sea posible. Además, hay diseños típicos para representar una función concreta y bien definida, y tienen un punto de entrada y un punto de salida. Por lo general, se trata de instrucciones cortas (es decir, de 50 a 100 instrucciones de longitud) y con un alto enfoque.

Uno de los elementos primordiales de programación que representan cada módulo es el subprograma o procedimiento. Un *subprograma*, es una serie de instrucciones para computadoras que ejecutan una función dada. Por lo común, se usan dos tipos de procedimientos: las *funciones* y las *subrutinas*; las primeras dan un solo resultado, mientras que las segundas ofrecen varios. Un *llamado o programa principal* invoca a esos módulos cuando sean necesarios. Por lo tanto, el programa principal organiza cada una de las partes en una serie lógica.

Además, se debe mencionar que mucha de la programación relacionada con paquetes de *software*, como Excel y MATLAB, echa mano del desarrollo de funciones. Es decir, los macros en Excel y las funciones de MATLAB se diseñaron para recibir alguna información, ejecutar cálculos y dar resultados. Así, el pensamiento modular es consistente con la manera en que se aplica la programación en el ambiente de estos paquetes.

El diseño modular tiene un gran número de ventajas. El uso de unidades breves e independientes constituye la lógica fundamental que tanto el programador como el usuario pueden seguir y entender con facilidad. El desarrollo es sencillo porque cada módulo se puede mejorar por separado. En efecto, en el caso de grandes proyectos, diferentes programadores pueden trabajar con partes individuales. En este mismo sentido, usted puede conservar su propia biblioteca de módulos para usarlos más tarde en otros programas. El diseño modular también hace que un programa sea cada vez más fácil de depurar y probar, de modo que los errores puedan ser detectados con facilidad. Finalmente, el mantenimiento y la modificación de los programas también se facilitan. Esto es fundamental porque los nuevos módulos pueden ser perfeccionados para cumplir cometidos adicionales, con lo cual se podrán incorporar a un esquema coherente y organizado.

2.2.3 El diseño top-down

Aunque en la sección anterior se presentó la noción de diseño modular, no aclaramos cómo reconocer los actuales módulos. El proceso de diseño de top-down es particularmente efectivo para lograr ese objetivo.

El *diseño top-down* es un proceso de desarrollo sistemático que empieza con una declaración general de los objetivos del programa para luego dividirse, sucesivamente, en segmentos más detallados. Así, el proceso de diseño va de lo general a lo específico. Muchos diseños top-down empiezan con una descripción literal del programa en sus términos generales. Esta descripción significa descomponer en unos cuantos elementos que definan las grandes funciones del programa; luego, cada uno de estos elementos se divide a su vez en subelementos. Este proceso debe continuar hasta que se hayan identificado bien todos los módulos.

Además de que proporciona un método para dividir el algoritmo en unidades bien definidas, el diseño top-down tiene otros beneficios. En especial, procura que el producto final sea cabalmente entendido. Si se comienza con una amplia definición y poco a poco se agregan los detalles, el programador no se verá en aprietos para examinar las **operaciones importantes**.

2.2.4 Programación estructurada

El modular y el *top-down* se acercan adecuadamente, por caminos efectivos, a lo que es la organización de un programa. En uno y otro caso, se hace hincapié en lo que debe hacer el programa. Aunque por lo común esto garantiza que el programa tendrá consistencia y estará hábilmente organizado, no puede asegurarse que las instrucciones del programa o código para cada módulo tengan alguna relación con la claridad y el orden. La programación estructurada, por otro lado, aborda la manera en que se formula el código del programa, de manera que sea correcto, modificable y fácil de entender.

En esencia, la *programación estructurada* es un conjunto de reglas que generan buenos hábitos de estilo para programar. Pese a que la programación estructurada es suficientemente flexible para permitir una creatividad considerable y formas de expresión personal, las reglas imponen tales restricciones que dan como resultado un código superior a las versiones no estructuradas. En especial, el producto terminado es más elegante y fácil de entender.

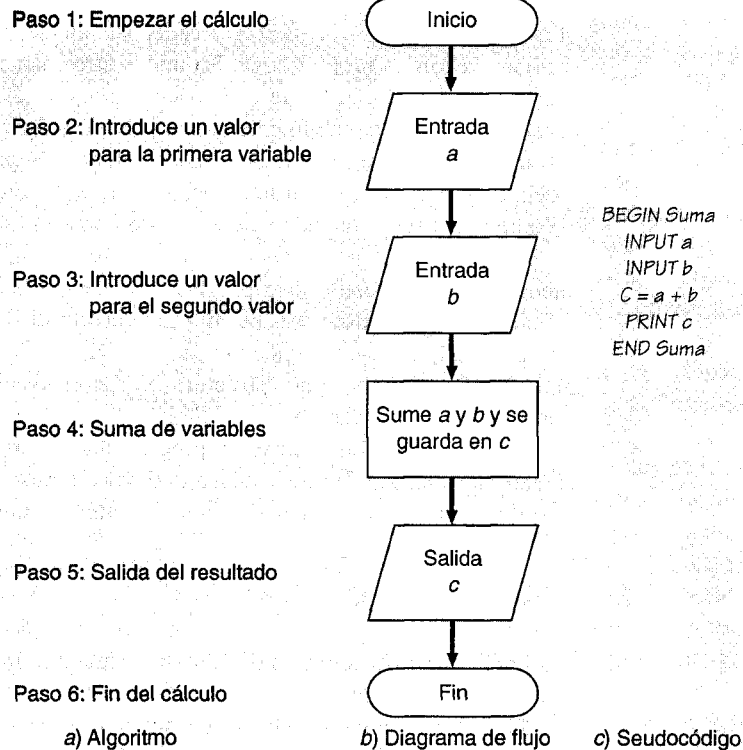
Las reglas que constituyen la programación estructurada serán descritas en la siguiente sección (2.4.2). Antes de hacerlo, volveremos al primer paso del proceso de desarrollo de programas, diseñando la lógica fundamental o el algoritmo del programa.

2.3 DISEÑO DE ALGORITMOS

Los problemas más comunes con que se encuentran los programadores inexpertos pueden estar relacionados con la preparación prematura de un programa que no tiene pies ni cabeza. En los inicios de la computación, esto era especialmente complicado porque los componentes de diseño y programación no estaban claramente separados. A menudo, los programadores debían presentar un proyecto de cómputo, sin antes formular un diseño claro, lo cual derivaba en toda clase de problemas e ineficiencias. Por ejemplo, el programador debía ponerse a trabajar sólo para descubrir que algunos factores preliminares clave habían sido descuidados. Además, era lógico que los programas concebidos de manera improvisada fueran invariablemente confusos. Esto quiere decir que para alguien era particularmente difícil modificar algo de ese programa. Hoy, debido a los altos costos por la preparación del *software*, se hace especial hincapié en que algunos aspectos —como una planeación preliminar— resultan ventajosos para obtener un producto más consistente y eficiente. El objeto de esta planeación es el diseño de *algoritmo*.

Un *algoritmo* es la secuencia de pasos lógicos necesarios para llevar a cabo una tarea específica, como la resolución de un problema. Para realizar este objetivo, un buen algoritmo debe contar con los siguientes atributos:

1. Cada paso debe ser determinado, es decir, no puede ser obra de la casualidad. Los resultados finales no pueden depender de quien sigue el algoritmo. En este sentido, un algoritmo es como una receta de cocina. Dos cocineros que trabajan cada uno por su lado con una buena receta deberían preparar platillos esencialmente idénticos.
2. El proceso debe ser siempre terminar después de un número finito de pasos. Un algoritmo no puede tener un final abierto.
3. El algoritmo debe ser lo suficientemente generalizado como para ocuparse de cualquier contingencia.

**FIGURA 2.4**

a) Algoritmo, b) Diagramas de flujo, y c) Seudocódigo de un programa para una simple suma.

La figura 2.4a muestra un algoritmo para resolver el problema sencillo de la suma de dos números. Dos programadores que trabajan cada cual por su lado a partir de este algoritmo podrían desarrollar programas con algún estilo diferente. No obstante, considerando los mismos datos, el programa debería dar resultados idénticos.

Las descripciones, literalmente paso a paso, de la forma que se representa en la figura 2.4a son una manera de expresar un algoritmo. Estas descripciones son particularmente útiles en el caso de problemas sencillos o para especificar las tareas esenciales de una larga programación. No obstante, para la representación detallada de programas complicados, resultan virtualmente inadecuadas. Por esta razón, se han desarrollado alternativas más versátiles, como los diagramas de flujo o los pseudocódigos.

2.3.1 Diagramas de flujo

Un *diagrama de flujo* es la representación gráfica de un algoritmo. Los diagramas de flujo emplean una serie de bloques y flechas, cada uno de los cuales representa una operación en especial o un paso del algoritmo. Las flechas representan la secuencia en que se llevan a cabo las operaciones. La figura 2.4b muestra el diagrama de flujo para un problema de la suma de dos números.

No todos los interesados en la programación de computadoras están de acuerdo con que los diagramas de flujo son un esfuerzo productivo. De hecho, hay programadores experimentados que no son partidarios de los diagramas de flujo. Sin embargo, creemos que hay tres buenas razones para estudiarlos. Primera, todavía se emplean para expresar y comunicar los algoritmos. Segunda, aunque no se empleen rutinariamente, en ocasiones serán útiles para proyectar, desenmarañar o transmitir la lógica de nuestros propios programas o de los de otras personas. Finalmente, y aún más importante para nuestros propósitos, son una excelente herramienta pedagógica. Desde el punto de vista de la enseñanza, los diagramas de flujo son un vehículo ideal para visualizar algunas estructuras fundamentales de control empleadas en la programación de computadoras.

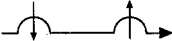
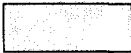
Símbolos de los diagramas. Nótese que, en la figura 2.4b, los bloques tienen diferentes formas para distinguir diferentes tipos de operaciones. En los incisos de la computación, los símbolos del diagrama de flujo no estaban estandarizados. Esto conducía a la confusión, porque los programadores usaban diferentes símbolos para representar la misma operación. Hoy, para facilitar la comunicación, se ha creado un número de sistemas estandarizados. En este libro usaremos el conjunto de símbolos mostrados en la figura 2.5.

El diagrama de flujo se lee de principio a fin, siguiendo las líneas de flujo de la lógica del algoritmo. Así, como se muestra en la figura 2.4b, deberíamos empezar con el block de "inicio" para luego seguir las flechas hasta llegar al block de "final". Si el diagrama de flujo está bien escrito, no debería haber duda con respecto a la ruta correcta.

Diseño de algoritmo top-down. No hay reglas para detallar cómo deberían ser los diagramas de flujo. Durante la elaboración de un algoritmo más grande, usted a menudo dibujará el diagrama de flujo con varios niveles de complejidad. A manera de ejemplo, la figura 2.6 representa una jerarquía de tres caracteres que podrían usarse en la elaboración de un algoritmo para determinar el grado de avance (GDA) o índice acumulativo de

FIGURA 2.5

Símbolos usados en diagramas de flujo.

SÍMBOLO	NOMBRE	FUNCIÓN
	Terminal	Representa el inicio y el final del programa.
	Líneas de flujo	Representa la lógica del flujo. Las curvas colocadas sobre las flechas horizontales indican que éstas pasan sobre las verticales sin concentrarse con ellas.
	Proceso	Representa cálculos o manipulación de datos.
	Entrada salida	Representa las entradas o salidas de datos e información.
	Decisión	Representa una comparación, pregunta o una decisión que determina las diferentes alternativas diferentes a seguir.
	Conector	Representa la confluencia de líneas de flujo.
	Conector de página	Representa una separación que continúa en la siguiente página.
	Ciclo de conteo controlado	Se usa para ciclos con un número específico de repeticiones.

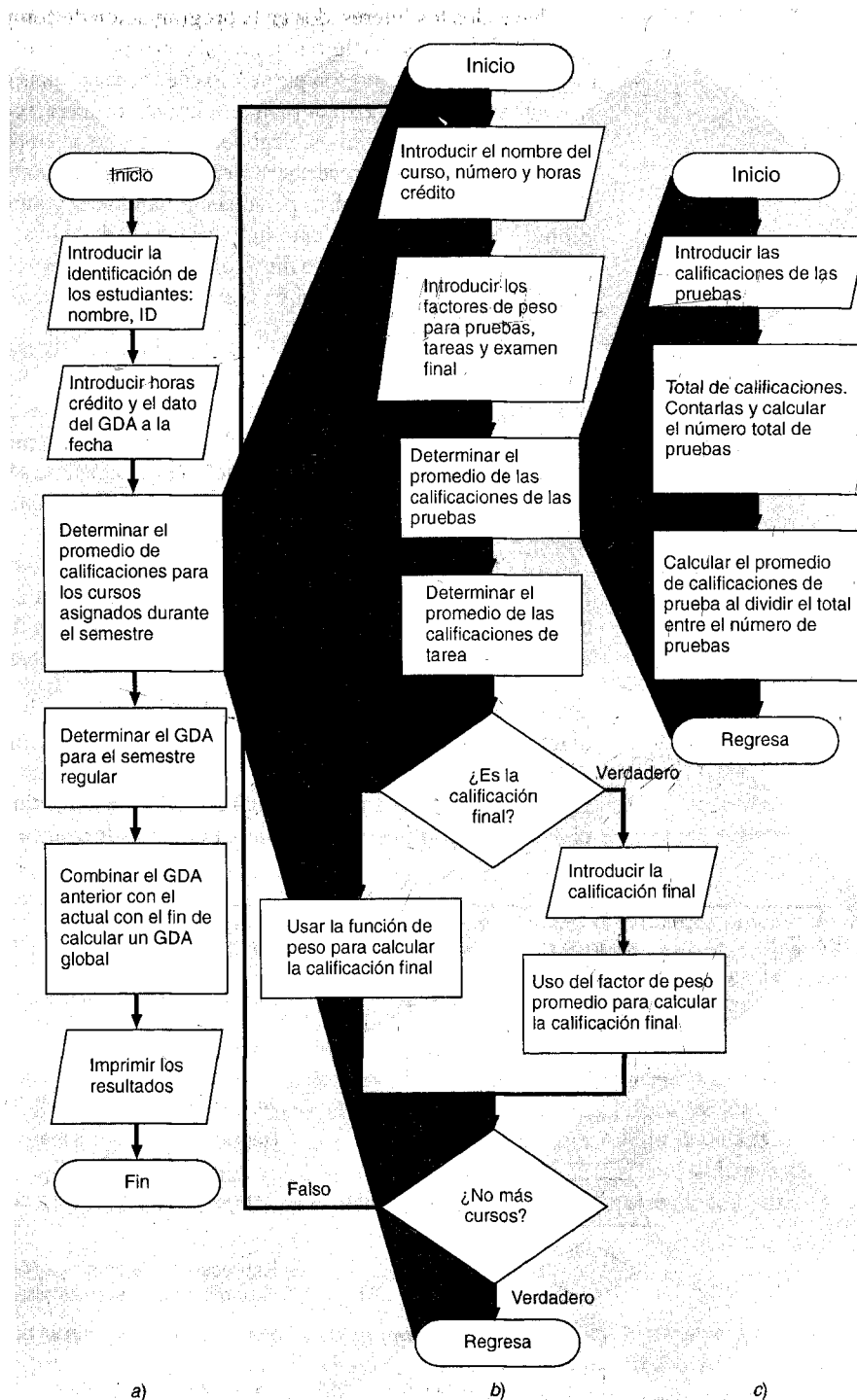
**FIGURA 2.6**

Diagrama de flujo por jerarquías para tratar el problema del promedio de calificaciones de los estudiantes. El sistema del diagrama de flujo a) proporciona una idea del plan. En b) se muestra con mayor detalle el diagrama de los grandes módulos. En c) se muestra un segmento de los grandes módulos con mayor detalle. El camino que sigue este problema se llama diseño top-down.

un estudiante. El sistema de diagrama de flujo (figura 2.6a) traza la figura grande. Ésta identifica las mayores tareas, o *módulos*, y la secuencia que se requiere para resolver todo el problema. Dicho diagrama de flujo es de valor incalculable para asegurar que el esquema total es lo suficientemente comprendido para ser exitoso. Luego, los módulos principales pueden separarse y trazarse con todo detalle (figura 2.6b). Finalmente, en ocasiones conviene separar en varios módulos para que la unidad sea más manejable, como en la figura 2.6c. Éste es un ejemplo del *proceso de diseño de top-down*, como se presentó en la sección 2.2.3.

En las siguientes secciones estudiaremos y perfeccionaremos el diagrama de flujo de la figura 2.6c. Esto nos enseñará a organizar los símbolos de la figura 2.5 para elaborar un algoritmo de manera eficiente y efectiva. En este caso, demostraremos cómo se puede reducir la potencia de las computadoras a tres tipos fundamentales de operaciones. La primera, que ya debería ser aparente, es que la lógica de la computadora puede proceder en una *secuencia* definida; como se muestra en la figura 2.6 incisos, a) y c), ésta se halla representada por un flujo lógico de bloque en bloque. Las otras dos operaciones fundamentales son la *selección* y la *repetición*.

Algoritmo estructurado. Obsérvese que las operaciones de la figura 2.6, incisos a) y c), ilustran una sencilla secuencia de flujo. Es decir, que las operaciones se hacen una después de otra. Sin embargo, obsérvese cómo el primer símbolo de toma de decisión en forma de diamante (figura 2.6b) permite que el flujo se divida, siguiendo una de las dos posibles rutas, dependiendo de la respuesta a la pregunta. En este caso, si se determinan las notas finales, el flujo seguirá la ruta de la derecha; pero si se determinan las notas intermedias, el flujo seguirá la ruta de la izquierda. A este tipo de operación de diagrama de flujo se le llama de *selección*.

Aparte de representar una bifurcación condicional, la figura 2.6b muestra otra operación de diagrama de flujo que incrementa notablemente la potencia de los programas de computadoras: la *repetición*. Adviértase que la bifurcación “falsa” en el símbolo de la segunda decisión permite que el flujo regrese al inicio del módulo y repita el proceso de cálculo para otro curso. Esta capacidad para representar tareas repetitivas está entre los poderes más importantes de las computadoras. A esta operación de “ciclo” regreso para repetir una serie de operaciones se le denomina *bucle* en la jerga de las computadoras.

Las operaciones de selección y repetición pueden usarse con gran ventaja cuando se desarrolla a los algoritmos. Esto puede demostrarse mediante la elaboración de una versión mejorada de la figura 2.6c, en la cual especificamos un diagrama de flujo total de notas de prueba y determinamos el promedio. Un diagrama de flujo con más detalles para lograr esos objetivos se presenta en la figura 2.7. Obsérvese que el algoritmo consiste en un ciclo y en construir una decisión. Los ciclos de entrada, las cuentas y las sumas de notas son operaciones que se repiten hasta que la entrada de una nota negativa acciona la salida del ciclo. En este punto, la selección construida sirve para calcular la nota promedio. La construcción de selección se emplea para estos propósitos y evita que se divida entre cero en caso de que se haya dado una nota no positiva.

Diagrama de flujo estructurado. Los símbolos del diagrama de flujo pueden servir para construir patrones lógicos de extrema complejidad. Sin embargo, los expertos en programación reconocen que, de no haber reglas, los diagramas de flujo pueden llegar a ser complicados, lo cual puede complicar más que facilitar la claridad de pensamiento.

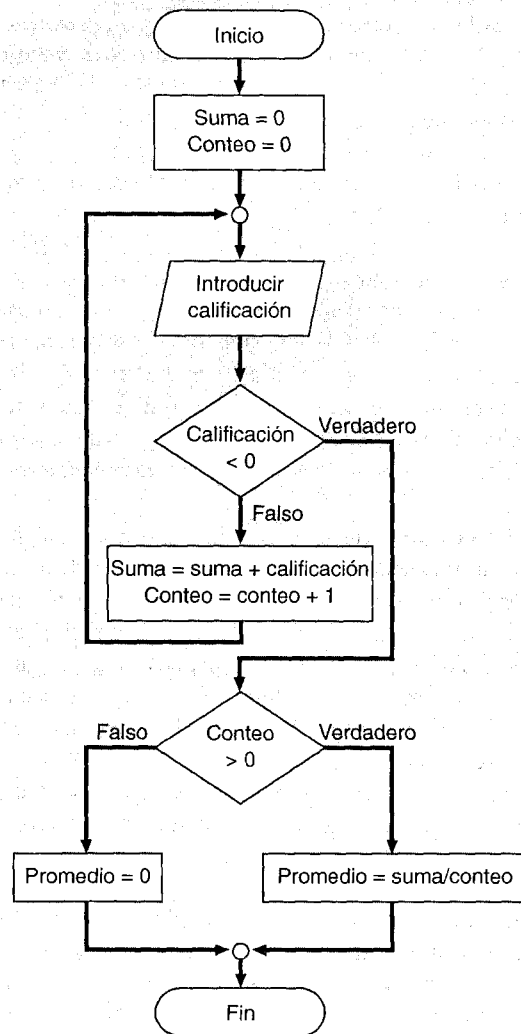
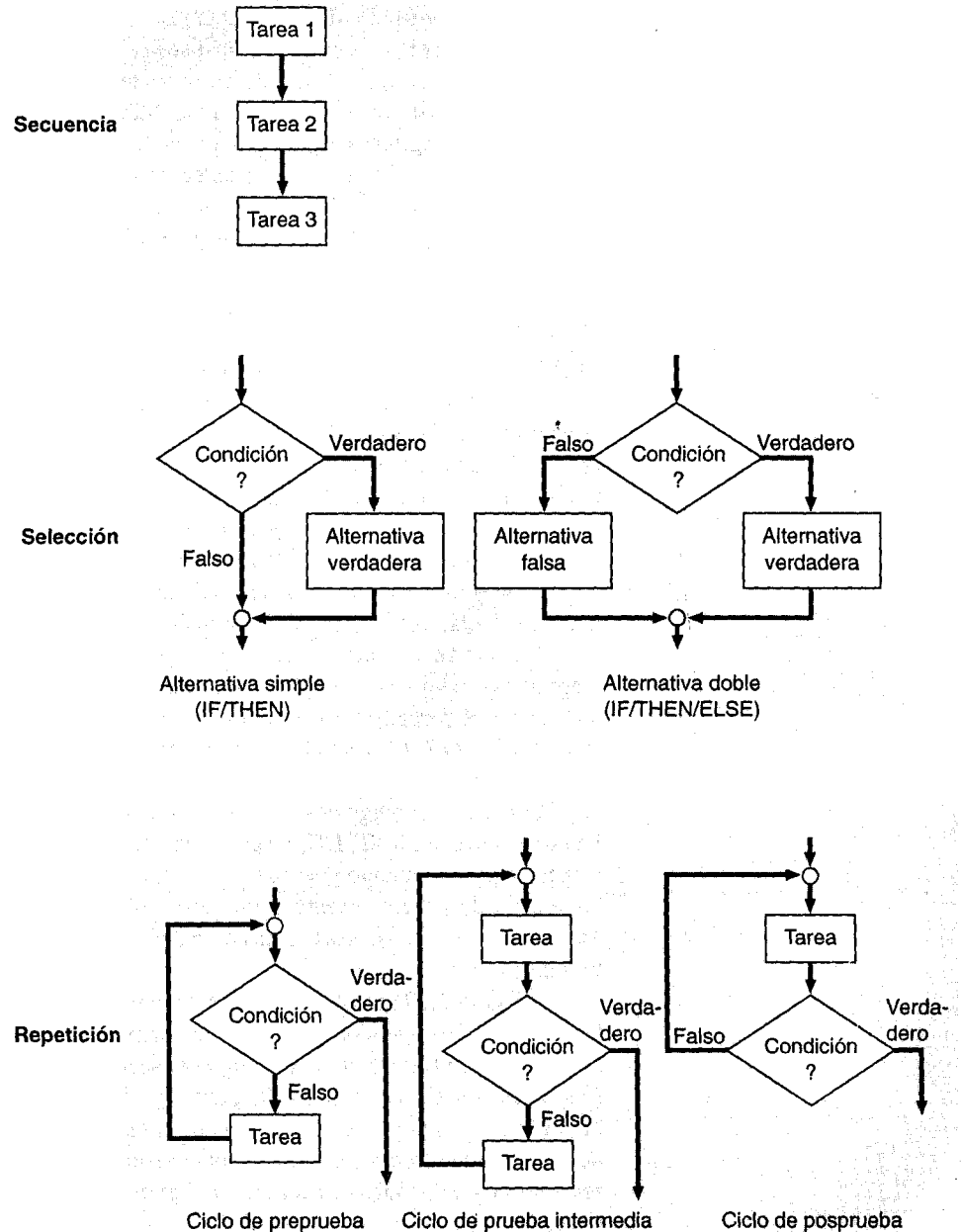
**FIGURA 2.7**

Diagrama de flujo detallado para calcular el promedio de las calificaciones de pruebas. Con el uso de la calculadora y el contador, este ejemplo mejora la versión mostrada en la figura 2.6.

Uno de los factores principales que han contribuido a la complejidad de los algoritmos es la transferencia incondicional, también conocida como GO TO. Como su nombre lo indica, esta estructura de control permite “ir a” cualquier otra parte en el algoritmo. En los diagramas de flujo esto se representa mediante una flecha. El uso indiscriminado de la transferencia incondicional puede dirigirnos a algoritmos que, a la distancia, nos recuerden un plato de espagueti: de ahí la denominación despectiva de *algoritmo espagueti*.

En un esfuerzo por evitar este dilema, los defensores de la programación estructurada han demostrado que cualquier programa puede ser elaborado con tres estructuras fundamentales de control, como se muestra en la figura 2.8, las cuales corresponden a tres operaciones (secuencia, selección y repetición) que vamos a tratar.

**FIGURA 2.8**

Las tres estructuras de control fundamentales.

Todos los diagramas de flujo deberían estar compuestos por tres estructuras fundamentales de control, como se advierte en la figura 2.8. Si nos limitamos a dichas estructuras y evitamos el GO TO, el algoritmo resultante será mucho más fácil de entender.

Antes de continuar, haremos una revisión práctica de nuestro dictamen sobre el "no GO TO". Por desgracia, algunos lenguajes (sobre todo el Fortran 77) y algunos paquetes no incluyen un conjunto completo de instrucciones estructuradas de programación. Esto es cierto, sobre todo, en el caso de los paquetes en que se han adoptado los lenguajes de programación primitivos y simplificados. Mientras más y más usuarios escriban macros y argumentos originales, esta situación cambiará. Sin embargo, mientras este libro esté en prensa, habrá ocasiones en que usted deberá usar el GO TO para llevar a la práctica una estructura de control estandarizada. Es decir, si usted puede evitar un ataque de nervios al usar el GO TO, todavía puede escribir un código decente en esos ambientes.

2.3.2 Seudocódigo

El objetivo de los diagramas de flujo es, obviamente, mejorar la calidad del programa de computación, lo cual consiste en una serie detallada de instrucciones llamada *código*. La revisión de las figuras 2.6 y 2.7 muestra cómo el diagrama de flujo top-down se mueve de acuerdo con el programa de computación. En referencia al diseño top-down, se crean diagramas de flujo más detallados mientras avanzamos desde los aspectos generales (figura 2.6a) hasta los módulos específicos (figura 2.6c). En su nivel más alto, el diagrama de flujo modular tiene casi la forma de un programa de computadora. Es decir, como se ve en la figura 2.7, cada bloque del diagrama de flujo representa una sola —pero bien definida— instrucción para la computadora.

Una alternativa para expresar un algoritmo que sea un puente de unión entre los diagramas de flujo y el código de computadora es el llamado *seudocódigo* el cual utiliza instrucciones parecidas a las de un código en lugar de los símbolos del diagrama de flujo. La figura 2.9 muestra las representaciones del pseudocódigo para las estructuras de control fundamentales.

Para el pseudocódigo de este libro, debemos adoptar algunas convenciones de estilo. Palabras clave como IF, DO, etcétera, escritas con letras mayúsculas, mientras que las etapas de procesamiento y las tareas en letras minúsculas. Adicionalmente, observe que las etapas de procesamiento están ensambladas. Así, las palabras clave forman un "sándwich" en torno a estas etapas para definir visualmente la extensión de cada estructura de control.

La figura 2.10 muestra la representación del pseudocódigo del diagrama de flujo de la figura 2.7. Esta versión gráfica es más semejante a un programa de computadora que el diagrama de flujo. Así, una ventaja del pseudocódigo es que es más fácil formular un programa con él, que con un diagrama de flujo. El pseudocódigo es también fácil de modificar y compartir con otros. No obstante, debido a su forma gráfica, los diagramas de flujo algunas veces son mejores para visualizar algoritmos complejos. En este libro se usará el pseudocódigo para escribir algoritmos relacionados con los métodos numéricos.

2.3.3 Otras estructuras útiles

Aunque se puede formular cualquier algoritmo numérico a partir de tres estructuras fundamentales de control, los expertos en computación han inventado varias estructuras adicionales de gran utilidad en métodos numéricos. Las más importantes son la estructura de selección multialternativa y el ciclo de conteo-bajo control.

FIG
Rep
seu
estr
func

FIG
Seuc
el pr
califi
Este
previ
del fl

Secuencia

Tarea 1
Tarea 2
Tarea 3

Selección

IF condición THEN
alternativa verdadera
END IF

IF condición THEN
alternativa verdadera
ELSE
Alternativa falsa
END IF

Alternativa simple
(IF/THEN)

Alternativa doble
(IF/THEN/ELSE)

Repetición

DO
IF condición EXIT
Tarea
END DO

DO
Tarea
IF condición EXIT
Tarea
END DO

DO
Tarea
IF condición EXIT
END DO

Ciclo de preprueba

Ciclo de prueba intermedia

Ciclo de posprueba

FIGURA 2.9

Representación del pseudocódigo de las tres estructuras de control fundamentales.

FIGURA 2.10

Seudocódigo para calcular el promedio de calificaciones de pruebas. Este algoritmo se presentó previamente en el diagrama del flujo de la figura 2.7.

Secuencia

Repetición

Comienza calificación promedio

suma = 0
conteo = 0

DO

INPUT calificación

IF calificación < 0 EXIT

suma = suma + calificación

conteo = conteo + 1

END DO

Selección

IF conteo > 0 THEN

promedio = suma/conteo

ELSE

promedio = 0

END IF

Fin calificación promedio

La Estructura de Selección Multialternativa. Supongamos que la cláusula ELSE de un IF/THEN/ELSE contiene otro IF/THEN, como en el siguiente pseudocódigo:

```
IF condición
  Tarea1
ELSE
  IF condición
    Tarea2
  ELSE
    Tarea3
  END IF
END IF
```

En tales casos, el ELSE y el IF pueden combinarse, así:

```
IF condición
  Tarea1
ELSE IF condición
  Tarea2
ELSE
  Tarea3
END IF
```

Note cómo el ELSE IF hace la estructura más concisa. No solamente se consolidan dos líneas, sino que también una de las declaraciones END IF ya no es necesaria. Un diagrama de flujo general para la estructura IF/THEN/ELSE IF se muestra en la figura 2.11.

Observe que como hay una cadena o “cascada” de decisiones. La primera es la declaración IF, y cada decisión sucesiva es una declaración ELSE IF. Al ir bajando, la primera condición encontrará que la verdadera prueba causará una bifurcación al bloque del código correspondiente, seguida de una salida de la estructura. Al final de la cadena de condiciones, si todas tienen pruebas falsas, habrá un bloque opcional ELSE.

El ciclo conteo-controlado. Los ciclos de la figura 2.9 se denominan *ciclos lógicos* porque terminan en una condición lógica. En contraste, los *ciclos de conteo-controlado* realizan un número específico de repeticiones o iteraciones. Desde luego, tales operaciones podrían ser programadas con un ciclo lógico. Por ejemplo, el siguiente pseudocódigo está concebido para repetir 10 veces:

```
i = 1
DO
  IF i > 10 EXIT
  PRINT i
  i = i + 1
END DO
```

La variable *i* es un contador que se mantiene en función del número de iteraciones. Si *i* es menor o igual a 10, la iteración se ejecuta. En cada paso, el valor de *i* se visualiza e *i* se

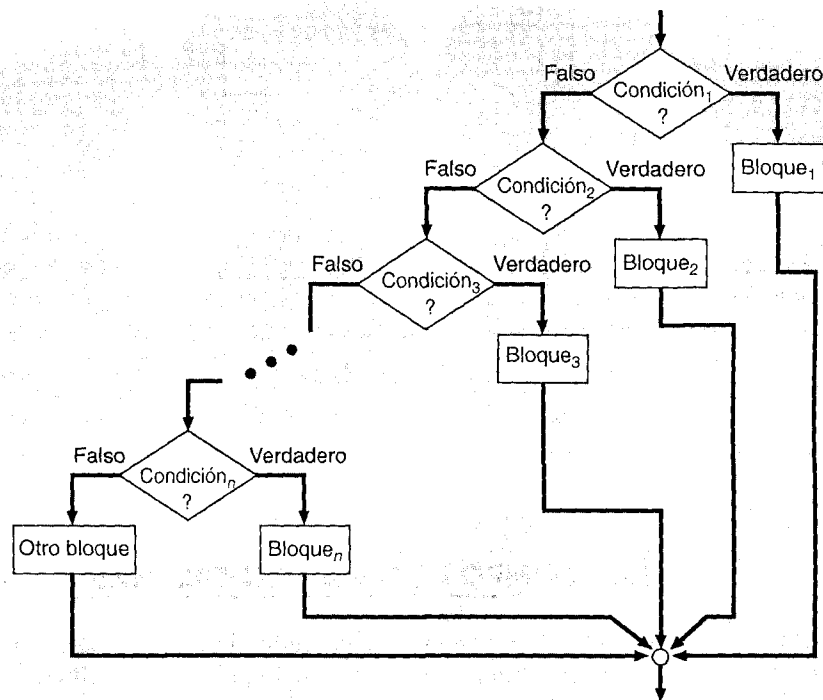


FIGURA 2.11

Estructura general de alternativa múltiple (IF/THEN/ELSE IF).

incrementa en 1. Después de la décima iteración, i vale 11; por lo tanto, la condición lógica será una prueba verdadera y el ciclo habrá terminado.

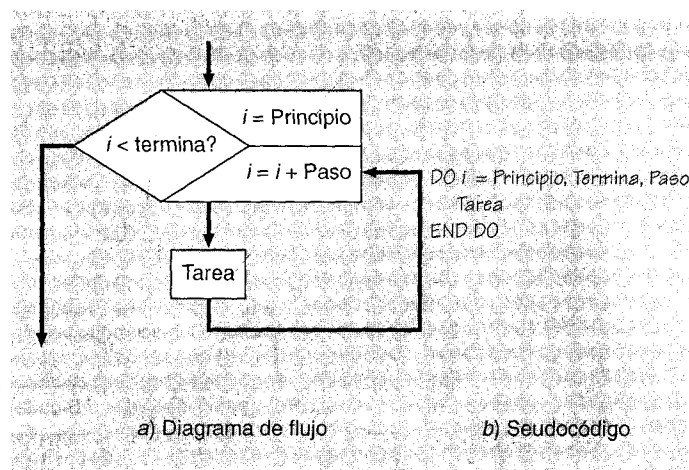
Aunque el ciclo anterior a la prueba es, en efecto, una opción factible para realizar un número específico de iteraciones, la operación es tan común que declaraciones de alto nivel, están disponibles en todo lenguaje de alto nivel común para llevar a cabo el mismo objetivo pero de una manera más eficiente. El diagrama de flujo y el pseudocódigo para la estructura de conteo-controlado se representan en la figura 2.12.

El ciclo de conteo-controlado trabaja como sigue: El *índice* (representado como i en la figura 2.12) es una variable que es un conjunto de valores iniciales de *principio*. El programa comprueba si el *índice* es menor o igual al valor *final*. De ser así, ejecuta la estructura del ciclo y los ciclos regresan a la declaración DO. Cada vez que se encuentra la declaración END DO, el *índice* se incrementa automáticamente por cada *paso*. Así, el *índice* actúa como un contador. Entonces, cuando el *índice* es mayor que el valor final (termina), la computadora transfiere automáticamente el control de salida del ciclo a la línea que sigue a la declaración END DO.¹

¹ Aunque creemos que la descripción anterior tiene un valor pedagógico, se debe hacer notar que un lenguaje de cómputo como el Fortran 90 en realidad no puede realizar preguntas para llevar a cabo ciclos de conteo-controlado. Más bien, el número de iteraciones es precalculado durante la compilación, razón por la cual es ilegal el cambio de valor de la variable que cuenta en la estructura del ciclo.

FIGURA 2.12

Diagrama de flujo y pseudocódigo de un ciclo de conteo-controlado que se puede realizar en métodos numéricos.



2.4 COMPOSICIÓN DE PROGRAMAS

Después de confeccionar el algoritmo, el siguiente paso es expresarlo como código. Antes de definir la manera de hacerlo, hablaremos sobre algunos lenguajes de computación.

2.4.1 Lenguajes de alto nivel y macros

Desde que empezó la era de la computadora, se han elaborado cientos de lenguajes de computación. Determinar cuál es el “mejor” lenguaje ha sido objeto de grandes debates. Por desgracia, muchos programadores tienen casi siempre un apego emocional a “su” lenguaje, e incluso llegan a asegurar que todos los otros son inferiores. Creemos que este tipo de actitudes cerradas pueden ser contraproducentes. Pese a que cada lenguaje tiene sus limitaciones, se puede usar con ventaja en un contexto de problemas particulares.

En este libro optaremos por el pseudocódigo para expresar nuestros algoritmos numéricos. Esto puede ser traducido con facilidad a los lenguajes principales para resolver problemas, usados por ingenieros y científicos: Fortran y C. Además, también se pueden aplicar en la elaboración de macros/guiones de paquetes tales como el Visual BASIC, dialecto usado por Excel.

Fortran. El Fortran, establecido como un *traductor de fórmulas*, fue presentado comercialmente por la IBM en 1957. Como su nombre lo indica, una de sus características es que emplea una notación que facilita la escritura de fórmulas matemáticas. Debido a ello, Fortran es un lenguaje natural de computación para muchos ingenieros y científicos.

Por otro lado, para las fórmulas matemáticas, Fortran tiene otras características relevantes para los métodos numéricos. Por ejemplo, tiene capacidad para trabajar con variables de doble precisión y manejar funciones especiales matemáticas, incluyendo variables complejas. Además, es altamente propicio para la programación modular porque tiene

subrutinas que permiten variables locales y la transferencia de valores entre los subprogramas y el del programa principal. Así, pueden elaborarse bibliotecas numéricas orientadas a los subprogramas y obtenerse comercialmente para usarse en un código Fortran.

C. Como se observa en la figura 2.3, la preparación de programas constituye en la actualidad una parte dominante de los costos en la computación. Creado en 1972 por los laboratorios Bell, C es actualmente el lenguaje más popular para el profesionalista que elabora programas.

C posee muchas de las características de un lenguaje de alto nivel y, por lo tanto, puede ser empleado por ingenieros para realizar cálculos numéricos. Aunque en términos generales es inferior al Fortran 90, también tiene acceso a bajos niveles de *hardware* y *software*, como es característico del lenguaje ensamblador. Al mismo tiempo, es más genérico que el lenguaje ensamblador, el cual es usualmente un *hardware* específico. Esta combinación de capacidad de bajo nivel y portabilidad es lo que hace al C atractivo para quienes elaboran programas.

Aunque los ingenieros y los científicos emplearon primeramente la computación para resolver problemas, una fracción pequeña pero significativa optó por elaborar sus programas. Además, algunas universidades prefirieron enseñar a los estudiantes de ingeniería y ciencias el C como primer lenguaje de computadora. Por ello, se debe reconocer que éste podría usarse, en la próxima década, por un buen número de ingenieros y científicos.

Visual BASIC. El BASIC fue creado originalmente por John Kemeny y Thomas Kurtz, a mediados de la década de los sesenta, como un lenguaje instructivo para los estudiantes del Dartmouth College. A partir de entonces, se fue sofisticando cada vez más (ejemplo de ello es el QUICK BASIC de Microsoft). En la actualidad, su principal implementación es el Visual BASIC. Este lenguaje está relacionado estrechamente con el sistema operativo de Windows de Microsoft, que es de hecho el ambiente de cómputo para las PC de IBM y equipos compatibles. Asimismo, es una de las herramientas principales en la elaboración de programas para PC. Además, debido a su función de macrolenguaje para el popular programa de hoja de cálculo Excel, su utilidad está más que reconocida.

Dialectos. El tiempo pasa y siguen creándose nuevas versiones o “dialectos” para cada lenguaje. A nadie sorprende que éstas incorporen lo mejor de las versiones anteriores. Los lenguajes específicos empleados en este libro —Fortran 90 y Excel Visual BASIC— fueron elegidos por su disponibilidad, porque son fáciles de usar y son compatibles, y porque tienen acceso a una programación estructurada. Así, todos los códigos utilizados en este libro están escritos en un estilo altamente estructurado que deriva de un pseudocódigo. En consecuencia, todos pueden ser traducidos con facilidad a otros lenguajes, como el Fortran 77 o el C. Además, pueden ser puestos en marcha sin mayores problemas con algunos paquetes de *software*, como las funciones del MATLAB. En la siguiente sección, explicaremos en detalle en qué consiste este acceso estructurado.

2.4.2 Programación estructurada

No todos están de acuerdo con la definición regular de programación estructurada. En general, la idea principal está comprendida en lo que podría llamarse el principio de estructura (Dijkstra, 1968):

El principio de estructura: La estructura estática (es decir, extendida fuera de esta página) de un programa correspondería, de una manera sencilla, a la estructura dinámica (es decir, extendida fuera de tiempo) de la computación correspondiente.

Así, el “salto alrededor de”, debido a una bifurcación indiscriminada y a otros malos hábitos en programación se podría evitar como consecuencia natural del ordenamiento de cálculos bien definidos.

Además del principio de estructura, hay otras reglas que pueden ser consideradas como parte de la filosofía de la programación estructurada. Las siguientes reglas generales están entre las más aceptadas:

1. Los programas deben consistir únicamente de tres estructuras fundamentales de control: secuencia, selección y repetición (indicadas en la figura 2.8). A decir de los expertos en computación, con estas estructuras básicas se puede elaborar cualquier programa.
2. Cada una de las estructuras debe tener sólo una entrada y una salida.
3. Hay que evitar las transferencias incondicionales (los GO TO).
4. Las estructuras se deben identificar claramente con comentarios y recursos visuales, como indentación, líneas en blanco y espacios en blanco.

Aunque las reglas pueden parecer engañosas, su apego a ellas puede evitar el código espagueti, que es el sello de las bifurcaciones indiscriminadas. Debido a sus beneficios, haremos hincapié en estas reglas y demás prácticas de la programación estructurada en las partes subsecuentes de este libro.

En las figuras 2.13 y 2.14 se muestra cómo se programa la construcción de la selección y la repetición en Fortran 90, C y Visual BASIC. Nótese que los códigos son muy parecidos alseudocódigo.

La figura 2.15 es un ejemplo categórico que contrasta entre lo estructurado y lo no estructurado. Los tres programas están concebidos para determinar el promedio de calificaciones, cuyo algoritmo está esbozado en la figura 2.10. Aunque los programas calculan valores idénticos, sus diseños tienen un marcado contraste. El propósito, la lógica y la organización del programa presentado en la figura 2.15a no es obvio. El usuario debe sondear en el código, línea por línea, para descifrar el programa.

En contraste, las partes fundamentales de la figura 2.15, incisos (b y c), se muestran inmediatamente. El programa se lleva a cabo en forma ordenada, de arriba abajo y sin mayores saltos. Cada segmento tiene una entrada y una salida, y cada uno está claramente delineado por espacios e indentación. En esencia, los programas se distinguen por una claridad visual que recuerda el buen diseño de un diagrama de flujo. Más aún, los programas estructurados son una evolución natural delseudocódigo que se muestra en la figura 2.10.

En la parte cargada los programas de la figura 2.15 (b y c), pueden tardar más tiempo en elaborarse que el programa de la figura 2.15a. Dado que muchos ingenieros pragmáticos están acostumbrados a “terminar su trabajo” de una manera eficiente y a tiempo, el esfuerzo extra no siempre estaría justificado. Tal sería el caso de un programa corto, que podría elaborarse “únicamente a un tiempo” de cálculo. Sin embargo, para programas más importantes que están destinados a alguna aplicación personal o de otro tipo, no hay duda de que una aproximación estructurada produciría beneficios a largo plazo.

Sudocódigo	Fortran 90	ANSI C	Visual BASIC
a) Decisión con alternativa simple			
IF Condición Alternativa verdadera END IF	IF (a < 0.) THEN b = -a END IF	if (a<0) { b = -a; }	IF a < 0. THEN b = -a END IF
b) Decisión con doble alternativa			
IF Condición Alternativa verdadera ELSE Alternativa falsa END IF	IF (a < 0.) THEN b = -a ELSE b = a END IF	if (a<0) { b = -a; } else { b = a; }	IF a < 0. THEN b = -a ELSE b = a END IF
c) Decisión con varias alternativas			
IF Condición Alternativa ₁ ELSE IF Alternativa ₂ ELSE Otra alternativa END IF	IF (a < 0.) THEN b = -a ELSE IF (a > 0.) THEN b = a ELSE b = 0. END IF	if (a<0) { b = -a; } else if (a>0) { b = a; } else { b = 0; }	IF a < 0. THEN b = -a ELSE IF a > 0. THEN b = a ELSE b = 0. END IF

FIGURA 2.13

Estructuras de selección elaboradas para Fortran 90, ANSI C y Visual BASIC.

2.5 CONTROL DE CALIDAD

Con las herramientas descritas anteriormente, usted debería ser capaz de elaborar un programa para realizar un cálculo en la computadora. Sin embargo, el hecho de que esté impresa la información del programa no es garantía que esas respuestas sean las correctas. En consecuencia, volveremos a los aspectos relacionados con la fiabilidad del programa y haremos hincapié en lo relacionado con la importancia del proceso de desarrollo de dicho programa. Ya que estaremos resolviendo problemas de ingeniería, mucho de nuestro trabajo será empleado directa o indirectamente por clientes y patrocinadores. Por esta razón, es esencial que nuestros programas sean confiables —esto es, que hagan lo que se supone deben hacer—. A lo mejor, la falta de confiabilidad puede ponernos en un aprieto. En el peor de los casos, para los problemas de ingeniería que incluyen la seguridad pública, esto sería una tragedia.

2.5.1 Errores o "basura"

Durante la preparación y ejecución de un programa de cualquier magnitud, es probable que haya errores. En ocasiones, a estos errores se les llama *basura* en la jerga de la computación. Este término fue acuñado por la almirante Grace Hopper, una de las pioneras.

Seudocódigo	Fortran 90	ANSI C	Visual BASIC
a) Ciclo lógico (prueba intermedia)			
DO Tarea IF Condición EXIT Tarea END DO	DO a=a/2. IF (a < 1.) EXIT a=a/4. END DO	while (1) { a=a/2; if (a<1) break; a=a/4; }	DO a = a/2. IF (a < 1.) EXIT a = a/4. LOOP
b) Ciclo de conteo controlado			
DO _i = Inicio, Final, Paso Tarea END DO	DO i = 1,10,2 x=x+i END DO	for(i=1;i<=10;i=i+2) { x=x+i; }	FOR i= 1 TO 10 STEP x=x+i NEXT i

FIGURA 2.14

Estructuras de repetición elaboradas para Fortran 90, ANSI C y Visual Basic.

ras en lenguajes de computación. En 1945, ella estaba trabajando con una sencilla computadora electromecánica y el aparato se apagó. Cuando ella y sus compañeras abrieron la máquina, descubrieron que unas polillas se habían alojado en uno de los transistores. Así, el término “basura” puede ser sinónimo de problemas y errores relacionados con las operaciones de la computadora, y el término *depurando* puede estar asociado con su solución.

Cuando elaboramos y corremos un programa, puede haber cuatro tipos de errores:

1. *Errores de sintaxis*, que violan las reglas del lenguaje, como la escritura correcta de palabras, la formación de números y cumplir otras convenciones. Estos errores son a menudo el resultado de escribir, por ejemplo, REED en lugar de READ. Esto usualmente resulta cuando el programa es compilado y resulta una impresión de salida con los mensajes de error. A dichos mensajes se les llama *diagnósticos* porque la computadora está ayudando a “diagnosticar” el problema.
2. *Errores de enlace o de construcción*, que ocurren durante la fase de enlace. Un ejemplo muy común podría ser cuando está mal escrito el nombre de una función intrínseca. En tales casos, la computadora podría imprimir un mensaje de error que diga: “Unresolved external reference” (función externa no resuelta).
3. *Run-time error (Errores durante la ejecución)*. Éstos ocurren durante la ejecución del programa. Un ejemplo es cuando hay, de entrada, un número insuficiente de datos para el número de variables de entrada de datos. En tales situaciones, se imprimirá un diagnóstico y usted deberá volver a dar los datos correctamente. En caso de otros errores durante la ejecución del programa se puede terminar la corrida o imprimir un mensaje con información acerca del error. En cualquier contingencia, usted deberá saber que ese error ha ocurrido.
4. *Errores lógicos*. Como su nombre lo indica, ocurren por fallas en la lógica del programa. Éstos son los peores errores, porque pueden ocurrir sin que haya un diagnós-

a) Fortran 77	b) Fortran 90	c) ANSI C
<pre> S=0. I=0 1 READ *, G IF(G. LT.0) GO TO 2 S=S+G I=I+1 GO TO 1 2 IF (I.EQ.0) GO TO 3 A=S/I GO TO 4 3 A=0. 4 PRINT *,A END </pre>	<pre> PROGRAM grader IMPLICIT none REAL :: avgrd PRINT *, avgrd() END FUNCTION avgrd () IMPLICIT none REAL :: sum,grade,avegrd INTEGER :: count count = 0 sum = 0. READ *, grade DO IF(grade < 0.) EXIT sum = sum + grade count = count + 1 READ *, grade END DO IF (count > 0) THEN avgrd = sum/count ELSE avgrd = 0. END IF END </pre>	<pre> main () { float avegrd (); printf ("%f\n", avegrd()); } float avegrd () { intcount; float sum, ave, grade; count = 0; sum = 0.0; scanf ("%f", &grade); while (1) { if (grade < 0.0) break; sum = sum + grade; count++; scanf ("%f", &grade); } if (count > 0) { ave = sum/count; } else { ave = 0.0; } return (ave); } </pre>

PROGRAM grader

FIGURA 2.15

Versiones del mismo programa en a) Fortran 77 no estructurado y estructurado, b) Fortran 90 y c) ANSI C. Todas las versiones hacen los mismos cálculos, pero difieren notablemente en cuanto a la claridad de los términos.

tico. Así, nuestro programa podría estar trabajando aparentemente bien en el sentido de ejecutar y generar resultados. Sin embargo, los resultados serán incorrectos.

Todo lo relacionado con estos cuatro tipos de error debe ser erradicado antes de emplear el programa para aplicaciones a ingeniería. Dividimos el proceso de control de calidad en dos categorías: depuración y prueba. Como ya se mencionó, la *depuración* implica corregir los errores detectados. En contraste, la *prueba* es un proceso que intenta detectar los errores que se desconocen. Adicionalmente a la prueba, también hay que determinar si el programa cumple con las necesidades para las que fue diseñado.

A este respecto haremos algunas aclaraciones: *depuración y prueba están interrelacionadas y a menudo se presentan una tras otra*. Por ejemplo, podríamos depurar un módulo al eliminar los errores obvios. Luego, después de una prueba, podríamos descu-

brir alguna basura adicional que necesitara corrección. Hay que repetir estos dos procesos hasta que estemos convencidos de que el programa es absolutamente confiable.

2.5.2 Depuración

Como ya se dijo, la depuración es para corregir errores conocidos. Hay tres caminos por los cuales será posible advertir esos errores. Primero, se puede establecer un diagnóstico explícito con la localización exacta y la naturaleza del error. Segundo, el diagnóstico puede establecerse, pero la localización exacta del error podría no ser clara. Por ejemplo, la computadora puede indicar que ha ocurrido un error en la línea donde hay una gran ecuación, incluyendo algunas funciones definidas por el usuario. Este error podría deberse a un error de sintaxis en la ecuación, o ser consecuencia de errores en la declaración al definir la función. Tercero, no hay diagnósticos pero el programa no corre de manera adecuada. Esto se debe por lo común a errores lógicos, como los ciclos infinitos, o a errores matemáticos que no tienen error de sintaxis. En el primer tipo de error, las correcciones son sencillas. Sin embargo, en los otros dos casos, hay que hacer alguna labor detectivesca para identificarlos y localizarlos con exactitud.

Por desgracia, la analogía con el trabajo de un detective es del todo acertada. Es decir, encontrar esos errores es a menudo difícil e implica mucho "trabajo de pie", reuniendo pistas y yendo por un callejón sin salida. En particular, como en la labor detectivesca, no existe una metodología clara. Sin embargo, existen algunos caminos por seguir para saber cómo hacer eficientes estos pasos.

Una clave para identificar la basura consiste en hacer impresiones intermedias de resultados. Con esta aproximación, a menudo es posible determinar en qué punto está mal el cálculo. En forma similar, el uso de una aproximación modular nos ayudará a localizar el error por este camino. Usted podría poner diferentes declaraciones de salida al principio de cada módulo y, por esta vía, enfocarse al módulo donde ocurrió el error. Además, siempre es bueno depurar (y probar) cada módulo por separado antes de integrarlo a todo el paquete. Hay que hacer notar que muchos ambientes de programación incluyen la construcción de herramientas para facilitar la búsqueda.

Además de los procedimientos anteriores, algunas veces la única opción será leer el código línea por línea siguiendo el flujo lógico y el resultado de todos los cálculos con lápiz, papel y calculadora: exactamente como un detective que en ocasiones se debe poner en el papel del criminal para resolver el caso. Algunas veces se tendrá que pensar como una computadora para depurar el programa.

Finalmente, no podemos dejar de subrayar que "una onza de prevención es mejor que una libra de curación". Es decir, sondear el diseño preliminar de un programa y las técnicas de programación estructurada son grandes caminos para evitar errores en un primer momento. Como en muchos otros aspectos de su vida, dentro de poco usted adoptará la filosofía de "pagar ahora y no después", desgracias que usted podrá experimentar como programador de computación.

2.5.3 Prueba

Uno de los más grandes errores de un programador novato es la creencia de que, si un programa corre e imprime resultados, es correcto. Un peligro peculiar son estos casos en que los resultados parecen "razonables", pero de hecho son incorrectos. Para asegurarse

de que esto no va a ocurrir, el programa debe estar sujeto a una serie de pruebas donde la respuesta correcta se conozca de antemano.

Como se mencionó en la sección anterior, es una buena práctica depurar y probar los módulos en general antes de integrarlos a la totalidad del programa. Así, por lo general las pruebas se realizan en varias fases. Hay pruebas de módulos, pruebas de elaboración, pruebas del sistema en conjunto y pruebas de operaciones.

Pruebas de módulos. Como su nombre lo indica, tratan sobre la fiabilidad de los módulos en forma individual. En vista de que cada uno está diseñado para realizar una tarea específica, una función bien definida, los módulos se pueden correr en forma aislada para determinar que su ejecución sea la correcta. Se puede elaborar una muestra de entrada de datos donde la salida correcta sea conocida. Estos datos pueden usarse al correr cada módulo y luego compararse con los resultados conocidos para verificar el desempeño correcto.

Pruebas de desarrollo. Se llevan a cabo cuando los módulos están integrados al programa total. Es decir, se presenta una prueba después de que se ha integrado cada uno de los módulos. Una manera eficaz de hacer esto es con una aproximación de top-down que comienza con el primer módulo y sigue en forma progresiva hacia abajo durante la ejecución del programa. Al realizar una prueba después de que se ha incorporado cada uno de los módulos, los problemas pueden ser aislados con mayor facilidad porque usualmente los nuevos errores se atribuyen al módulo que se añadió al último.

Después de que se ha ensamblado el programa total, puede estar sujeto a una serie de *pruebas totales del sistema*. Esto se debería planear para probar el programa con 1) datos típicos, 2) datos poco comunes pero válidos, 3) datos incorrectos con el fin de corroborar la capacidad del programa para manejar los errores.

Todas las pruebas anteriores son frecuentemente llamadas *pruebas-alpha*. Una vez que resultan exitosas, usualmente se inicia la fase de las *pruebas operacionales* o *beta*. Estas pruebas se realizan para corroborar qué tan realista es el ambiente en que se presenta el programa. Una forma común de hacerlo consiste en que algunas personas (incluyendo al último usuario) pongan en operación el programa. Algunas veces, las pruebas de operación descubren la basura y, además, proporcionan información sobre la manera en que se puede perfeccionar el programa, de modo que se conozcan más de cerca las necesidades del usuario.

Si bien el objetivo último de los procesos de prueba es asegurar que los programas estén libres de error, habrá casos complicados en que sea imposible subordinar un programa a una probable contingencia. Asimismo, se deberá admitir que el nivel de las pruebas dependerá de la importancia y la magnitud del contexto del problema por el cual se está aplicando el programa. Es obvio que se deberán aplicar distintos niveles de rigor al probar un programa para determinar los promedios de un equipo de softbol, en comparación con otro programa que regula la operación de un reactor nuclear o una estación espacial. Sin embargo, en cada caso se debe realizar la prueba apropiada que sea consistente con el contexto del problema en particular.

2.6 DOCUMENTACIÓN Y MANTENIMIENTO

2.6.1 Documentación

Una vez que el programa se ha depurado y probado, debe documentarse. La documentación es la descripción, en español, que permite al usuario utilizar el programa en forma

más sencilla. Recuerde que, como las otras personas que emplean su programa, usted es un "usuario". Aunque el programa se puede ver con claridad y sencillez cuando se aplica por primera vez, con el tiempo ese mismo código se puede desordenar. Por lo tanto, se debe incluir la documentación adecuada para que usted o cualquier usuario entiendan inmediatamente el programa y la manera en que se debe aplicar. Esta tarea presenta dos aspectos, la parte interna y la parte externa.

Documentación interna. Imagine, por el momento, un texto con puras palabras y sin párrafos, subtítulos o capítulos. Resultaría muy complicado estudiar en un libro así. El modo en que están estructuradas las páginas —es decir, todos los dispositivos que actúan para separar y organizar el material— hace que un texto sea más eficiente y atractivo. En ese sentido, la documentación interna del código de la computadora puede mejorar sustancialmente el programa para que el usuario pueda entenderlo y sepa cómo trabaja.

Las siguientes son algunas sugerencias generales para documentar internamente los programas:

1. Incluya en el módulo, al principio del programa, el título, nombre y fecha de éste. Es decir, su firma, el sello del programa y la manera en que trabaja.
2. Incluya un segundo módulo para definir cada una de las variables clave.
3. Elija nombres variables que reflejen el tipo de información que las variables están guardando.
4. Inserte espacios entre las declaraciones para hacer más fácil su lectura.
5. Acostumbre hacer abundantes comentarios a lo largo del programa para agregar explicaciones y saltarse líneas con el fin de etiquetar, dar claridad y separar los módulos.
6. En particular, haga comentarios para etiquetar y hacer resaltar todos los módulos.
7. Use indentación para clarificar la estructura del programa; en especial la indentación puede usarse para los ciclos y las conclusiones.

Documentación externa. Se refiere a las instrucciones en forma de mensajes de salida e impresión suplementaria. Estos mensajes se presentan cuando el programa corre y se piensa en realizar una salida atractiva o amigable al usuario. Esto implica el uso efectivo de espacios, líneas en blanco o caracteres especiales para ilustrar la secuencia lógica y la estructura de la salida del programa. Una salida atractiva simplifica la detección de errores y resalta la comunicación de los resultados del programa. Finalmente, el programa puede ser ideado para dar salida a los mensajes descriptivos de error, alertar e informar al usuario sobre posibles problemas.

La cuestión de una impresión suplementaria puede ir desde una simple hoja hasta un manual de usuario. El manual de usuario para la computadora ejemplifica lo que es una completa documentación. Este manual dice cómo correr el sistema en la computadora y los programas de disco operativo. Cuando desarrollamos nuestros programas, podemos encontrar útil elaborar un breve "manual de usuario" que proporcione una descripción e instrucciones del programa.

2.6.2 Memoria y mantenimiento

Recuerde que cuando apague su computadora, sus archivos activos pueden destruirse. Para conservar un programa que va a usar más tarde, debe transferirlo a un dispositivo de memoria secundaria, como un disco o una cinta, antes de apagar la máquina.

Aparte del acto físico de conservar un programa, la memoria y el mantenimiento consisten en dos grandes tareas: 1) actualizar el programa en función de la experiencia, y 2) asegurar que el programa esté en una memoria segura. Lo anterior es materia de comunicación con el usuario para que en forma respetuosa considere estas sugerencias para un mejoramiento de sus diseños. La pregunta de la memoria de seguridad de los programas es especialmente crítica durante la aplicación. Debería crear siempre una copia de respaldo antes de hacer mayores cambios al código. También, es imperativo mantener un respaldo en dispositivos externos, como un diskette.

2.7 ESTRATEGIA DE PROGRAMACIÓN

Uno de los primeros objetivos al usar este libro debe ser el desarrollo de una librería de métodos numéricos. Si se tiene en mente construir pequeños macros o grandes y complicados programas de computadora, dicha librería tendrá un inmenso valor en el futuro académico y el producto profesional. Será necesario hacer diversos arreglos en los medios de la computación para lograr este objetivo, lo cual incluye el pseudocódigo y los programas.

El pseudocódigo y otras descripciones de los algoritmos se proporcionan, por muchos métodos, en este libro. Si se les usa como base para sus programas, esto puede aumentar su capacidad para comprender los métodos numéricos.

Además de la propia colección de programas del usuario, se ha desarrollado un software genérico llamado Métodos Numéricos Toolkit, que se emplea junto con este libro. El paquete incluye módulos dedicados a cinco de los métodos numéricos fundamentales (tabla 2.1). Cada programa de métodos numéricos es totalmente ilustrado con un ejemplo, a propósito de cada capítulo del libro. Los ejemplos pueden proporcionar modelos para el diseño de pantallas de entrada y salida, incluyendo una muestra gráfica de resultados. Los ejercicios para resolver en casa se incluyen en los capítulos pertinentes para reforzar la capacidad de usar el disco en su computadora. Además, el programa puede usarse para corroborar la precisión de sus propios programas.

TABLA 2.1 Los programas en versión WINDOWS del paquete de métodos numéricos TOOLKIT.

Programa	Capacidad
Raíces de ecuaciones	Determina las raíces reales de una simple ecuación algebraica o trascendente mediante el método de bisección. Puede también ser usada para hacer gráficas de funciones.
Sistemas de ecuaciones lineales algebraicas*	Resuelve o encuentra la inversa de hasta 20 ecuaciones lineales algebraicas mediante la descomposición LU implantada en la eliminación gaussiana
Ajuste de curvas*	Ajusta los datos a una regresión polinomial hasta de décimo orden y caracteriza estadísticamente la adecuación del ajuste.
Integración*	Integra funciones o datos tabulados espaciados en forma desigual con el método del trapecio.
Sistemas de ecuaciones diferenciales ordinarias*	Resuelve hasta cinco ecuaciones diferenciales ordinarias simultáneas usando el método de Runge - Kutta de cuarto orden.

*Indica un mejoramiento comparado con la versión anterior del DOS.

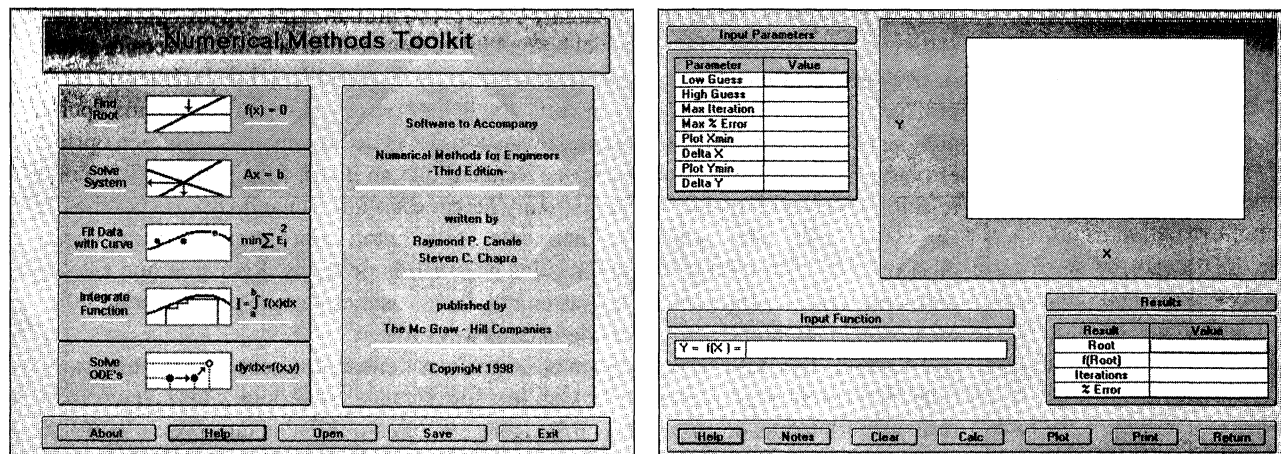
EJEMPLO 2.1 Instalación y uso del paquete TOOLKIT de métodos numéricos en su computadora

Enunciado del problema. El propósito de este ejemplo es usar el programa incluido en el texto (TOOLKIT de métodos numéricos).

Solución. Correr el programa `install.exe` que viene en el disco incluido en el texto y comenzar con el TOOLKIT. En la pantalla debe aparecer el menú principal del TOOLKIT, como se muestra en la figura 2.16a. Primero, observe el renglón con cinco botones de control en la parte baja del menú. El botón About (acerca de) describe al programa y da crédito a los autores del programa. Oprima este botón y luego oprima OK. El botón Help (ayuda) tiene cinco mensajes en secuencia, que recuerdan ciertas convenciones, y las características de diseño de TOOLKIT. El primer mensaje de ayuda describe las funciones del usuario reconocidas por el programa. Éstas incluyen las funciones trigonométricas básicas y funciones exponenciales. Tome en cuenta que los argumentos de las funciones trigonométricas son expresadas en radianes y no en grados. Cuando use el TOOLKIT el seno de 90 grados debe ser escrito $\text{SIN}(2 \cdot \pi \cdot 90/360)$ o $\text{SIN}(\pi/2)$ o $\text{SIN}(1.57)$. El segundo mensaje de ayuda describe cómo generar las gráficas por medio del paquete TOOLKIT. Se necesitan cuatro parámetros para construir una gráfica. Éstos son los valores mínimos de X y Y, y el intervalo entre las marcas para X y Y. Están siempre los intervalos de 10 Delta X y 10 Delta Y en un diagrama, de manera que los valores máximos de X y Y son $X_{\text{min}} + 10 \text{ Delta X}$ y $Y_{\text{min}} + 10 \text{ Delta Y}$. Esta convención le permitirá crear gráficas a su gusto. El tercer mensaje de ayuda explica en qué consiste el botón rojo de advertencia empleado por TOOLKIT. Como usted verá, TOOLKIT está diseñado para mostrar todas las entradas, así como los resultados numéricos y gráficos para cada método en una simple pantalla, lo cual permitirá que usted vea, con una simple mirada, todos los aspectos de un problema en particular. Sin embargo, suponga que quiere dibujar la función $Y(X)$ con un rango de "0 a 10" de X y Y. Al inspeccionar la gráfica usted decidirá si es necesario cambiar el rango de Y de "0 a 10" a "0 a 20". Cuando cambie el valor de 10 a 20 en la tabla de datos, usted notará que el botón de gráfica cambia del negro al rojo. Esta llamada de atención indica que la gráfica debe actualizarse para reflejar el nuevo rango de 20. Al apretar el botón rojo de Gráfica éste cambiará a negro, señalando que la gráfica y los parámetros de entrada son consistentes. Así, cuando use el programa sobre una base interactiva, asegúrese de que no haya botones rojos cuando vaya a guardar los resultados finales. El cuarto mensaje de ayuda señala cómo los cálculos del TOOLKIT se pueden salvar y recuperar como archivos con la extensión ".nmt". Así, si se emplea el TOOLKIT como ayuda para resolver una tarea relacionada con un curso de ingeniería mecánica, podría ponerle a su archivo el nombre de `MEPROB4.nmt`. El quinto mensaje de ayuda advierte que las funciones de impresión de TOOLKIT podrían no ser las adecuadas para trabajar con algunas configuraciones de la red.

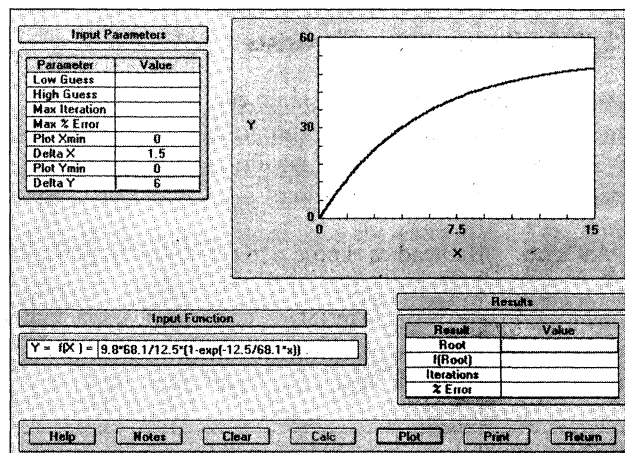
Los botones de Open (abrir) y Save (salvar) son como las clásicas ventanas de Windows, que permiten nombrar, almacenar y volver a llamar los archivos que fueron elegidos. Cualquier análisis realizado con el TOOLKIT se puede perder al salir, a menos que use el botón de Save para guardar.

A continuación, observe los cinco grandes botones en el lado izquierdo del menú principal. Estos botones tienen nombres, figuras y símbolos matemáticos para recordar las funciones de los métodos numéricos. Hay que explorar el programa entre cada uno de esos botones en las secciones apropiadas del texto.



a)

b)



c)

FIGURA 2.16

a) Título en pantalla del programa de métodos numéricos TOOLKIT. b) Pantalla de la localización de la raíz mostrando las gráficas de la velocidad de caída del paracaidista [ecuación (1.10)].

Por ahora usaremos el Find Root o búsqueda de raíces (botón superior) del programa para dibujar la velocidad de descenso del paracaidista como una función del tiempo. Al oprimir el botón de Find Root, la pantalla mostrará un patrón similar al de la figura 2.16b. La pantalla tiene cinco partes principales: una tabla con los parámetros de entrada, una ventana para las funciones de entrada, una ventana para gráficas, una tabla de resultados y una barra que contiene los botones de control. Hay que hacer caso omiso de la capacidad del programa para encontrar raíces, hasta la parte dos, y simplemente use el programa para graficar la ecuación (1.10) con $m = 68.1$ kg, $c = 12.5$ kg/s y $g = 9.8$ m/s². La figura 2.16c muestra los resultados para Xmin y Ymin = 0 con Delta X = 1.5 y Delta Y = 6. Hay que hacer la prueba con varios valores para los parámetros de la

gráfica, incluyendo negativos en Xmin y Ymin para familiarizarse con las operaciones y el diseño de la gráfica. Nótese que el botón Plot es rojo cuando la gráfica necesita actualizarse (por ejemplo, la gráfica que se muestra en la pantalla y los parámetros de la gráfica son inconsistentes). Experimente con el botón Clear (limpiar) como una opción para cambiar los parámetros de entrada y las funciones de las ventanas. También oprima el botón Notes (Notas). Aparecerá una ventana que se podrá usar para escribir notas relacionadas con el trabajo. Podrá guardar y llamar estas notas cuando use los botones de Save y Open del menú principal. Finalmente, use el botón Print (imprimir) para generar una copia de la gráfica y la función y los parámetros de datos. Recuerde que esto no podrá trabajar con algunas impresoras en red. Puede usar esta copia para hacer un informe sobre un cliente, o como parte de la tarea asignada.

La habilidad del TOOLKIT para crear gráficas tiene tantos usos como el usuario se proponga como meta para entender y aplicar los métodos numéricos y los cálculos para resolver problemas de ingeniería. Investigaremos estas utilidades tanto como otras capacidades del TOOLKIT en las secciones apropiadas del texto.

2.7.1 Paquetes y librerías

A lo largo de este libro, se hará un esfuerzo por describir cómo poner en operación los métodos numéricos mediante programas y librerías. Los primeros hacen referencia a Mathcad, Excel y MATLAB, las últimas a la librería IMSL. A continuación, se da una pequeña descripción de cada uno.

Mathcad. Mathcad es el producto estrella de MathSoft, Inc. El paquete de software es un puente que une a quienes gustan de las hojas de cálculo como Excel y garabatear en un block, lo que da como resultado una hoja de trabajo interactiva que permite al usuario realizar tareas de matemáticas, obtener datos de primera mano y elaborar gráficas propias de la ciencia y la ingeniería. La información y las ecuaciones son los datos para la hoja de trabajo, es como si se trabajara con papel y lápiz más que con un lenguaje de programación o con notación de hoja de cálculo. La primera versión del Mathcad fue presentada en 1986, y la séptima en 1997.

El Mathcad puede realizar tareas tanto en modo numérico como en modo simbólico. En el modo numérico, las funciones del Mathcad y los operadores dan una respuesta numérica, mientras que en el modo simbólico los resultados se dan como expresiones generales o ecuaciones. Maple V, un paquete de matemáticas de tipo simbólico, es la base del modo simbólico y fue incorporado al Mathcad en 1993.

El Mathcad tiene varias funciones y operaciones que permiten llevar a cabo, en forma conveniente, muchos de los métodos numéricos desarrollados en este libro. Tales métodos se describirán en detalle en los siguientes capítulos. En virtud de que es fácil de usar, creemos que el Mathcad es particularmente útil como herramienta pedagógica.

Excel. Es una hoja de cálculo producida por Microsoft, Inc., cuyo tipo especial de software matemático permite al usuario dar y realizar cálculos en renglones y columnas de datos. Por ello, hay una versión computarizada de una hoja de trabajo de grandes cuentas, en la que se pueden realizar y destacar grandes cálculos conectados entre sí. Debido a que se puede actualizar cuando se cambia cualquier número en la hoja, la hoja de cálculo es ideal para tipos de análisis “¿qué sucedería si...?”